Semantics and Multi-Query Optimization Algorithms for the Analyze Operator

Marios Iakovidis [0009−0006−9061−3450]
University of Ioannina, Ioannina 45110, Greece
miakovidis@cs.uoi.gr

Panos Vassiliadis [0000−0003−0085−6776]
University of Ioannina, Ioannina 45110, Greece
pvassil@cs.uoi.gr

June 8, 2026

A short version of this paper has been accepted at DOLAP 2026:
M. Iakovidis, P. Vassiliadis. Multi-Query Optimization for the novel Analyze Operator. Accepted at 28th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP) co-located with EDBT/ICDT 2026, Tampere, Finland - March 24, 2026.

Abstract

In their hunt for highlights, i.e., interesting patterns in the data, data analysts have to issue *groups* of related queries and *manually* combine their results. To the extent that the analyst goals are based on an *intention* on what to discover (e.g., contrast a query result to peer ones, verify a pattern to a broader range of data in the data space, etc), the integration of *intentional* query operators in analytical engines can enhance the efficiency of these analytical tasks. In this paper, we introduce, with well-defined semantics, the *ANALYZE operator*, a novel cube querying intentional operator that provides a 360^o view of data. We define the semantics of an ANALYZE query as a tuple of five internal, facilitator cube queries, that (a) report on the specifics of a particular subset of the data space, which is part of the query specification, and to which we refer as the *original query*, (b) contrast the result with results from peer-subspaces, or *sibling queries*, and, (c) explore the data space in lower levels of granularity via *drill-down queries*. We introduce formal query semantics for the operator and we theoretically prove that we can obtain the exact same result by merging the facilitator cube queries into a smaller number of queries. This effectively introduces a *multi-query optimization (MQO)* strategy for executing an ANALYZE query. We propose three alternative algorithms, (a) a simple execution without optimizations (*Min-MQO*), (b) a total merging of all the facilitator queries to a single one (*Max-MQO*),

and (c) an intermediate strategy, *Mid-MQO*, that merges only a subset of the facilitator queries. Our experimentation demonstrates that Mid-MQO achieves consistently strong performance across several contexts, Min-MQO always follows it, and Max-MQO excels for queries where the siblings are sizable and significantly overlap.

1 Introduction

Routinely, data analysts find themselves issuing *groups* of related queries in an attempt to uncover hidden gems in the data. Such significant findings, or *highlights*, are practically verifications of the existence of interesting properties (e.g., a trend, a peak, an outlier, etc) in a subset of the data that is of interest. Automating the extraction of highlights is, therefore, an important task in accelerating the work of data analysts, and allowing for more extensive and complete exploration of the data space. To the extent that the underlying analyst goals are based on an *intention* on what to discover, (e.g., contrast a query result to results of queries exploring subsets of the data space that are similar to the one under scrutiny, verify a pattern to a broader range of data in the dataspace, forecast the trend etc) we have introduced the need for *intentional* query operators [VM18, VMR19] that abstract these data explorations into higher level operators. As such, these operators should come with a formal model for data, formal semantics, and, of course, with optimization methods for their execution.

Related work on the subject of *Automated Highlight Extraction* provides tools for multidimensional data exploration to discover interesting data patterns [Sar99, SAM98, WSZ+20, AKS+21, MDHZ21]. The primary focus of these tools is to explore a data space, find data patterns, and measure how interesting they are using interestingness metrics. However, the research landscape is characterized by (a) the absence of a widely accepted formal model that encompasses the proposed tasks in a coherent framework (the Intentional Model of [VM18, VMR19] is a first attempt), (b) at large, the omission of the opportunity to utilize multidimensional hierarchical data spaces (in the Business Intelligence sense), by focusing almost solely on relational data, and (c) to the best of our knowledge, the absence of an operator that provides an all-encompassing view of the data of interest to an analyst in a single invocation.

In this paper, we propose the *ANALYZE operator*, a novel cube query operator that formalizes the intention of providing a 360^o view of the data to the data analyst, via a single invocation. Being an algebraic operator, ANALYZE comes with (a) a well-defined multidimensional, hierarchical underlying data model with data cubes and dimensions as its basic elements [Vas22], (b) well defined semantics on the expected results, and also, in our case, (c) optimization strategies for its efficient execution. The simplicity of a hierarchical multidimensional model facilitates a simple query operator that exploits the model's ability to compute sibling, ancestor and descendant values at different levels of coarseness smoothly. To facilitate comprehension, we will use the following query, as a reference example throughout the paper:

ANALYZE	$sum(Store\ Sales)$ as $SumSales$
FROM	$Sales$
FOR	$Date.Quarter\ =\ 1997-Q3\ \wedge$
	$Customer.state\ =\ 'CA'\ \wedge$
	$Promo.Media\ =\ 'Daily\ Paper'$
GROUP BY	$month,\ customerRegion$
AS	$PaperPromoCA1997Q3$

The semantics of an ANALYZE query include five internal, *facilitator* cube queries. First, the operator has to deal with the information requested for a very specific subset of the data space that is obtained via (a) a set of filters and (b) a pair of groupers that are used in a GROUP-BY fashion to aggregate a measure. We call this the *original query* of the operator as it targets a subset of the data space of interest. In our example here, the three filters on 1997-Q3, CA and Daily Paper constrain the data involved to a subspace of interest. The result groups data by month and customer region and reports the sum of store sales. Second, we want to contrast the result with similar, peer results. To this end, we use the combination of grouper and filter conditions to obtain *sibling* subspaces of the data that query subspaces sharing the same ancestor values with the filters of the original query. In our case, since we want to constrain time to 1997-Q3, we need to find similar quarters to 1997-Q3 to contrast the results to. This is where the hierarchical nature of the multidimensional space kicks in, by allowing us to define as siblings the values sharing the same ancestor value: assuming that we group quarters in years, we contrast the original query to the quarters of the year 1997, to which 1997-Q3 belongs. Similarly, to find the peers of CA, we contrast CA to the rest of the states of the United States. Finally, we explore the data space in lower levels of granularity via what we call *drill-down queries* (very much in the traditional OLAP sense), and report the sum of store sales for the months of the quarter of 1997-Q3 and the customers residing in CA. We would like to stress here that the emphasis is on the algebraic, operator side and not on the language side: an ANALYZE query could be produced via a CLI call, an SQL-like expression, or a point-n-click interface, yet they would all end-up in the execution of the same algebraic operator.

Apart from introducing formal query semantics for the operator, however, we have been able to theoretically prove that we can obtain the exact same result by merging the facilitator cube queries into a smaller number of merged queries that exploit cube usability results to reduce redundant computation. This theoretical result effectively allows us to introduce a *multi-query optimization (MQO)* strategy that merges several collaborator queries into one or more merged queries, in an attempt to speed up the execution of an ANALYZE query. In fact, we have devised several strategies of Multi-Query Optimization at merging the underlying facilitator queries into one. In the simplest case, no optimization is performed and the five facilitator queries are executed independently: we refer to this as *Min-MQO*, as it introduces the least degree of query merging. Second, we follow the theoretical result and merge all the

facilitator queries into a single one, a strategy that we call *Max-MQO*, as it implements the original theoretical result of merging everything. Finally, based on our original experimental observations that identified cases of low performance for the aforementioned strategies, we also introduce an intermediate strategy, *Mid-MQO*, that strikes a balance between the two extremes and merges only a subset of the facilitator queries. In all merging strategies, once the results of the merged queries are obtained, they are post-processed, such that the exact facilitator queries are populated correctly.

We have extensively evaluated the performance of each MQO strategy via various query workloads on multiple datasets, to assess both the effect of data size and query selectivity to the execution cost as well as the optimal strategy. Our experiments demonstrate that *Mid-MQO* is the most efficient algorithm that scales smoothly with data size and selectivity and typically achieves the best performance. Although in many cases *Max-MQO* has the worst performance, when the sibling queries are sizable and significantly overlap, *Max-MQO* is the fastest algorithm; we can predict via a decision tree when this is the case. Finally, *Min-MQO* is never the optimal strategy, although it follows *Mid-MQO* fairly closely.

The remainder of this paper is structured as follows. Section 2 reviews related work in exploratory data analysis and pattern discovery. Section 3 introduces the formal modeling background. Section 4 presents the design and semantics of the ANALYZE operator. Section 5 develops our multi-query optimization strategies. Section 6 reports our experimental evaluation. Section 7 concludes with a discussion of the findings and future directions.

2 Related Work

Automated EDA. Exploratory Data Analysis (EDA) [MS20] allows data analysts to interact with dataset management tools to gain highlights. The goal of EDA is the production of highlights, in order to find interesting, surprising and important facts of a data subspace (likely a query result) and present them using data narration methods [Sar99, SAM98, Sar00, SS01, IPC15, THY+17, EMS19, DHX+19, WSZ+20, MDHZ21, PAB+21, SXS+21, BRH+22, AMP23, SCC+23, LYZ+23, MDW+23, Ame24, XWJ24, LMS+25]. Over the years, several operators have been proposed to complement the fundamental ones. The *DIFF* operator [Sar99] returns the set of tuples that most successfully describe the difference of values between two cells of a cube that are given as input. The same author also describes a method that profiles the exploration of a user and uses the Maximum Entropy principle to recommend which unvisited parts of the cube can be the most surprising in a subsequent query [Sar00]. Finally, the *RELAX* operator allows to verify whether a pattern observed at a certain level of detail is present at a coarser level of detail too [SS01].

Alternative operators have also been proposed in the Cinecubes method [GV13, GVM15]. The goal of this effort is to facilitate automated reporting, given an original OLAP query as input. To achieve this purpose two operators

(expressed as *acts*) are proposed, namely, (a) *put-in-context*, i.e., compare the result of the original query to query results over similar, sibling values; and (b) *give-details*, where drill-downs of the original query's groupers are performed.

Intentional Model. The Intentional Model [VM18, VMR19] envisions Business Intelligence tools, where users will apply intentional operators over data, to simplify the querying step of data analysis operations. The user expresses high-level requirements, like 'analyze', 'assess', 'predict', that have to be addressed via auxiliary queries, ML models, highlights and data stories. (a) auxiliary queries that collect related data; (b) models, in the ML sense of data that extract patterns from the collected data, (c) highlights, interesting parts of the data and models, ranked with respect to data interestingness, and (d) data stories, presenting the findings via data visualization and storytelling methods. Although the general framework of the intentional model is specified in [VM18, VMR19], the exact implementation of the operators is an open problem (to which we currently contribute with respect to the ANALYZE operator). [FGM$^+$23, FGM$^+$21] explore the ASSESS operator for contrasting query results to specified 'benchmarks' of expected performance. [FMPR22] presents the DESCRIBE operator which generates cubes annotated with model components (e.g., clusters, outliers). [FRM24] suggests an EXPLAIN operator, which uses statistical models to explain why a measure takes certain values in relation to other measures. The most similar work to the current one, and the root of the Intentional Model, is the Cinecubes method [GV13, GVM15], which produces a data story as a PowerPoint presentation with results of auxiliary drill-down and sibling queries.

Multi-Query Optimization (MQO).[Sel88] establishes the foundational evidence that processing multiple queries together yields considerable cost reductions compared to independent query execution. Sellis presents algorithms that search the space of possible combined execution plans, trading off the cost of materializing shared subplans against the benefits of reused computation, and develops heuristics to keep the evaluation cost manageable for realistic workloads. Following up on the aforementioned work, [SG90] provides one of the first formal treatments of MQO, as the task of generating an optimal execution strategy for a set of concurrent queries by identifying and exploiting common subexpressions, shared join paths, and reusable intermediate results. Cost-model extensions are introduced for evaluating shared plans, enabling the optimizer to reason about materialization and reuse. [RS09] discusses a comprehensive experimental validation for Volcano optimizers. [HRK$^+$09, LKDL12] provide extra rewriting techniques. [KS15] provides approximate optimization techniques. [RS09] discusses comprehensive experimental validation that demonstrates significant performance improvements with acceptable optimization overhead through the proposed Volcano-based heuristic algorithms. Volcano-based algorithms are designed to make MQO computationally efficient and extensible to complex queries and cost models. The work discusses algorithmic primitives that decompose MQO into tractable subproblems (e.g., bounded-cost search, grouping of queries by similarity) and combines them into an optimizer that scales to larger workloads compared to prior exhaustive approaches. [HRK$^+$09]

proposes a rule-based approach that takes advantage of declarative transformation rules to detect and restructure shared computations between queries. The rule-based design merges equivalent subexpressions, factors out common operators, and introduces shared materialized results where cost. [WC13] reports performance improvements over state-of-the-art techniques in MapReduce frameworks. The authors formulate techniques to identify common computations across multiple MapReduce jobs. In this context, MapReduce jobs are rewritten or scheduled so that shared map or reduce stages are executed once, and their outputs are reused across jobs. [LKDL12] introduces techniques that identify common subgraph patterns, such as join motifs, and leverages them to build unified execution plans capable of serving many SPARQL queries simultaneously. The authors propose structural indexing and graph-pattern clustering methods to reduce the combinatorial explosion in multi-query plan enumeration. Beyond performance gains, [KS15] advances the theoretical foundation by providing the first approximation factor guarantees for MQO algorithms. The authors propose cost-aware grouping and plan-construction techniques that exploit structural properties of query workloads, enabling the discovery of beneficial shared subexpressions without exhaustive enumeration.

Comparison to the State of the Art. The main contribution of this work is the formal introduction of a novel operator along with its optimization techniques. Although conceptually related, our work here does not pertain to the core of the area of multi-query optimization: our MQO algorithms do not explore alternative query execution plans, but rather exploit theoretical guarantees of correctness. Compared to the most similar line of work, [GV13, GVM15], we change the query semantics to address efficiency issues (thus we face a new problem), provide formal semantics of an operator and, and most importantly, we introduce multi-query optimization strategies for the speed up of query processing. Compared to the rest of the corpus of related literature, to the best of our knowledge, there is no other work with multi-query optimization strategies for a formal, single operator that provides a 360^o view of the data with opportunities for optimization exactly due to the integration of various queries.

3 Preliminaries & Formal Background

In our deliberations, we assume the formal model of [Vas22] (also used in [Vas23]) for the definition of the multidimensional space, cubes and cube queries.

3.1 Multidimensional Space

Multidimensional space. Data are defined in the context of a multidimensional space. The multidimensional space includes a finite set of dimensions. Dimensions provide the context for factual measurements and will be structured in terms of dimension levels, which are abstraction levels that aid in observing the data at different levels of granularity. For example, the dimension *Time*

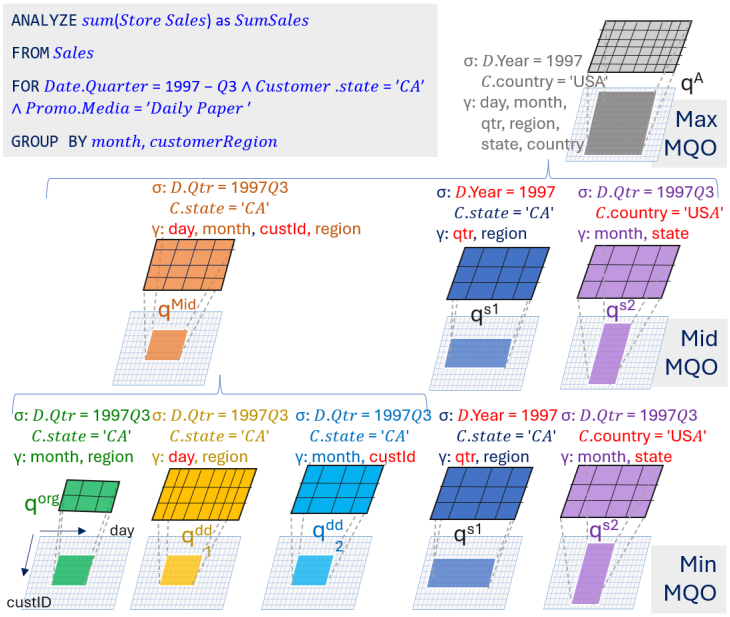


Figure 1: Facilitator queries per algorithm

is structured on the basis of the dimension levels *Day*, *Month*, *Year*, *All*. A *dimension level* L includes a name and a finite set of values, $dom(L)$, as its domain. Following the traditional OLAP terminology, the values that belong to the domains of the levels are called *dimension members*, or simply *members* (e.g., the values Paris, Rome, Athens are members of the domain of level *City*, and, subsequently, of dimension *Geography*). A *dimension* is a non-strict partial order of a finite set of *levels*, obligatorily including (a) a most detailed level at the lowest possible level of coarseness, and (b) an upper bound, which is called *ALL*, with a single value 'All'. We denote the partial order of dimensions with $\preceq$, i.e., $D.L_{low} \preceq D.L_{high}$ signifies that $D.L_{low}$ is at a lower level of coarseness than $D.L_{high}$ in the context of dimension D – e.g., $Geo.City \preceq Geo.Country$. To ease notation, unless explicitly mentioned otherwise, in the sequel, we will assume total orders of levels, which means that there is a linear order of the levels starting from the lower level and ending at ALL with a linear chain of precedence.

We can map the members at a lower level of coarseness to values at a higher level of coarseness via an *ancestor function* $anc_{L^l}^{L^h}()$. Given a member of a level L_l as a parameter, say v_l, the function $anc_{L^l}^{L^h}()$ returns the corresponding ancestor value, for v_l, say v_h, at the level L_h, i.e., $v_h = anc_{L^l}^{L^h}(v_l)$. The inverse of an ancestor function is not a function, but a mapping of a high level value to a set of *descendant values* at a lower level of coarseness (e.g., *Continent* Europe

is mapped to the set of all European cities at the *City* level), and is denoted via the notation $desc_{L^h}^{L^l}()$. For example $Europe = anc_{City}^{Continent}(Athens)$. See [Vas22] for more constraints and explanations. The union of the domains of all levels of a dimension will be abusively called the domain of the dimension, $dom(D)$. The multidimensional space is generated from the Cartesian product of the domains of its dimensions, thus each point c in the multidimensional space carries the coordinates $c(c_1, \ldots, c_n)$, with exactly one value for each dimension.

Cubes. Facts are structured in cubes. A *cube* C is defined with respect to several dimensions, fixed at specific levels and also includes a number of *measures* to hold the measurable aspects of its facts. Thus the schema of a cube is a set of attributes, including a set of dimension levels (over different dimensions) and a set of measures that include factual measurements for the data stored in the cube.

Formally, the schema of a cube $schema(C)$, is a tuple, say $[D_1.L_1, ..., D_n.L_n, , M_1, ..., M_m]$, or simply $[L_1, ..., L_n, M_1, ..., M_m]$, with the combination of the dimension levels (each coming from a different dimension) acting as primary key and context for the measurements and a set of measures as placeholders for the (aggregate) measurements.

If all the dimension levels of a cube schema are the lowest possible levels of their dimension, the cube is a *detailed cube*, typically denoted via the notation C^0 with a schema $[D_1.L_1^0, ..., D_n.L_n^0, M_1^0, ..., M_m^0]$. The result of a query q is a set of cells that we denote as $q.cells$. Each record of a cube C under a schema $[D_1.L_1, \ldots, D_n.L_n, M_1, \ldots, M_m]$, also known as a *cell*, is a tuple $c = [l_1, \ldots, l_n, m_1, \ldots, m_m]$, such that $l_i \in dom(D_i.L_i)$ and $m_j \in dom(M_j)$. The vector $[l_1, \ldots, l_n]$ signifies the *coordinates* of a cell. Equivalently, a *cell* can be thought as a point in the multidimensional space of the cube's dimensions annotated with, or hosting, a set of measures. A cube includes a finite set of cells as its extension.

Overall, a cube database includes both hierarchies and cubes as distinct but related first-class citizens. A cube is defined with respect to levels that belong to the hierarchies, and, therefore, its position in the multidimensional hierarchical space constructed by the hierarchies is determined by these levels.

3.2 Cube Queries

Queries. A cube query is a cube too, specified by: (a) the detailed cube over which it is imposed, (b) a selection condition that isolates the facts that qualify for further processing, (c) the grouping levels, which determine the coarseness of the result, and, (d) an aggregation over some or all measures of the cube that accompanies the grouping levels in the final result.

$$q = < C^0, \phi, [L_1, ..., L_n, M_1, ..., M_m], [agg_1(M_1^0), ..., agg_m(M_m^0)] >$$

We assume selection conditions which are conjunctions of atomic filters of the form $L = value$, or in general $L \in \{v_1, \ldots, v_k\}$.

Selection conditions of this form can eventually be translated to their *equivalent* selection conditions at the detailed level, via the conjunction of the detailed

equivalents of the atoms of ϕ. Specifically, assuming an atom $L \in \{v_1, \ldots, v_k\}$, then $L^0 \in \{desc_L^{L^0}(v_1) \cup ... \cup desc_L^{L^0}(v_k)\}$, eventually producing an expression $L^0 \in \{v'_1, ..., v'_{k'}\}$ is its detailed equivalent, called *detailed proxy*. The reason for deriving ϕ^0 is that ϕ^0, as the conjunction of the respective atomic filters at the most detailed level, is directly applicable over C^0 and produces exactly the same subset of the multidimensional space as ϕ, albeit at a most detailed level of granularity. For example, assume $Year \in \{2018, 2019\}$, its detailed proxy is $Day \in \{2018/01/01, \ldots, 2019/12/31\}$. For a dimension D that is not being explicitly filtered by any atom, one can equivalently assume a filter of the form $D.ALL = all$.

We also assume aggregation functions agg_i include aggregate functions like $\{sum, max, min, count, ...\}$ with the respective well-known semantics.

Cube Query Semantics. The semantics of the query are: (i) apply ϕ^0, the detailed equivalent of the selection condition over C^0 and produce a subset of the detailed cube, say q^0, known as the detailed area of the query; (ii) map each dimension member to its ancestor value at the level specified by the grouping levels and group the tuples with the same coordinates in the same same-coordinate group; (iii) for each same-coordinate group, apply the aggregate functions to the measures of its cells, thus producing a single value per aggregate measure.

A cube query is also a cube under the schema $[L_1, ..., L_n, M_1, ..., M_m]$ and with the cells of the query result (denoted as $q.cells$) as its extension.

Constraints. In the context of this work, we imply several assumptions that mainly serve the purpose of clarity. First, we work with cube queries that involve a single measure. Second, we assume strictly two aggregator levels for the result, and third, for all atomic filters, we assume that if the dimension with a filter is also a grouper, the atomic filter is expressed in a level greater than or equal to the grouper.

- We work with cube queries that involve a single measure
- We assume strictly two aggregator levels for the result
- For all the atomic filters, we assume that, if the dimension with a filter is also a grouper, the atomic filter is expressed in a level greater or equal than the grouper. We will denote with $D.L^\sigma$ the level of the atomic filter ($D.L^\sigma = v$) and with $D.L^\gamma$ the respective grouper, and require that if such a pair exists in the query definition, then $D.L^\gamma \preceq D.L^\sigma$

The rationale behind the above constraints is as follows:

- To avoid overloading notation, we opt to work with a single measure for the purpose of clarity.It is straightforward to generalize our results to multiple measures by joining single-measured cube query results over their dimension levels.
- We also opt to work with exactly 2 grouper levels. This is a traditional, reasonable constraint that stems from the presentational constraint of having to present results in a 2D screen, either in a tabular form, or a chart.

- The constraint on the relationship of filters and groupers refer to the notion of perfect rollability of [Vas22]. For example, assuming we are grouping results by *city*, if a filter is applied over the *Geography* dimension it should involve a specific *city*, or a *country* (whose measures we group by city), or a *continent*. Had we filtered for a certain *neighborhood*, and then requested to group by city would (a) produce results, although (b) with at least unclear semantics, and, in any case (c) violating the perfect rollability requirement. Hence, we believe the constraint is reasonable and useful.

Then, we will employ the following query expression:

$$q = < C^0,\ \phi,\ [L_\alpha, L_\beta, M],\ agg(M^0) >$$

4 The Analyze Operator

In this section, we introduce the ANALYZE operator in terms of semantics, syntax and execution strategies.

4.1 Syntax & Semantics

Assume the setup already discussed in Section 3, with a detailed cube C^0 defined over a list of hierarchical dimensions. The syntax of the ANALYZE operator is as follows:

ANALYZE	$agg(M^0)$ as M FROM C^0
FOR	ϕ
GROUP BY	L_α, L_β
AS	$queryName$

with the variables participating in the definition having the obvious semantics already introduced in Section 3.

To complement this SQL-like definition, we will also employ an algebraic representation of the operator, as $analyze(C^0, \phi, [D_\alpha.L_\alpha, D_\beta.L_\beta, M], agg(M^0))$, or equivalently as:

$$analyzeOp = \langle\ C^0,\ \phi,\ [D_\alpha.L_\alpha,\ D_\beta.L_\beta, M],\ agg(M^0)\ \rangle$$

Example. To exemplify our discussions, we will employ a cubefied version of the well-known Foodmart schema [Hyd25]. Foodmart includes a cube, Sales with 3 measures, contextualized by 5 dimensions, namely Product, Date, Promotion, Customer, and Store, each with its own levels.All dimensions include the level All as the uppermost level of coarseness.The query presented in Section 1 will be used as our reference example.

Semantics. The semantics of the operator involve the introduction of three separate groups of intermediate cube queries, which we call facilitator queries, with each group producing a set of distinct cube query results. Assuming a given ANALYZE query AQ, the query set of $AQ.queries$ is a triplet of query

sets $< Q^{org}, Q^{sib}, Q^{dd} >$ and the result of AQ, $AQ.result$ is a triplet with the results of the respective queries in the three query sets.

The 3 query sets that the operator produces are as follows:

- (a) Q^{org} is a singleton set, comprising exactly one query q^{org}, to which we will refer as *the original query*, that computes the aggregate value for M^0 as specified by the variables of the operator.
- (b) Q^{sib} is a set of sibling queries, whose goal is to compare the behavior of key values in the operator definition against the behavior of their peer values.
- (c) Q^{dd} is a set of drill-down queries, whose goal is to provide more detailed data to the analyst for the values produced by the original query.

Figure 1 demonstrates the queries of the reference example, along with the auxiliary queries that the subsequent MQO algorithms will produce. In the sequel, we present the semantics and rationale of each of these auxiliary, facilitator queries.

4.1.1 Sibling queries

The key word concerning siblings is *context*: Sibling queries aim to compare (a) the different slices of the data space that pertain to the original query against (b) their peers, in order to put them in context. By comparing data slices to their peer, the data slices are effectively assessed by the analyst and, thus, contextualized.

Original idea. Intuitively, given an original query, a sibling query differs from it as follows: instead of an atomic selection formula $L_i = v_i$ of the original query, the sibling query contains a formula of the form $L_i \in children(parent(v_i))$.

Formally, assume an original query q^{org}

$$q^{org} = \langle\ C^0,\ \phi_\alpha \wedge \phi_\beta \wedge \phi_\square,\ [D_\alpha.L^\gamma_\alpha,\ D_\beta.L^\gamma_\beta, M],\ agg(M^0)\ \rangle$$

with the following constraints:

- ϕ_α an optional selection atom of the form: $D_\alpha.L^\sigma_\alpha = v_\alpha$, $L^\gamma_\alpha \preceq L^\sigma_\alpha$
- ϕ_β an optional selection atom of the form: $D_\beta.L^\sigma_\beta = v_\beta$, $L^\gamma_\beta \preceq L^\sigma_\beta$
- $\phi_\square$ an optional conjunction of atoms for dimensions other than the groupers D_α, D_β, with all its atoms ϕ_i being of the form, ϕ_i: $L_i = v_i$

Observe that for each grouper dimension we discriminate between the grouper level (e.g., L^γ_β) and the filtering level, e.g., L^σ_β. In our example, we group by L^γ_β: *state*, whereas our L^σ_β level is *country* ($country = 'USA'$), at a higher level than the grouper.

In its simplest possible form, a sibling query of q^{org} with respect to dimension say D_α, would be

$$q^{s^{\alpha}} = \langle\, C^0,\, \phi^{\star}_{\alpha} \wedge \phi_{\beta} \wedge \phi_{\square},\, [D_{\alpha}.L^{\gamma}_{\alpha},\, D_{\beta}.L^{\gamma}_{\beta}, M],\, agg(M^0)\, \rangle$$

with $\phi^{\star}_{\alpha}$ an atom of the form: $D_{\alpha}.L^{\sigma}_{\alpha} = v^{\star}_{\alpha}$, under the constraint that $anc^{L^{\sigma}_{\alpha+1}}_{L^{\sigma}_{\alpha}}(v_{\alpha}) = anc^{L^{\sigma}_{\alpha+1}}_{L^{\sigma}_{\alpha}}(v^{\star}_{\alpha})$.

In other words, we replace the selection value of exactly one atom with a sibling value (expressed by the constraint that their mother-value, one level higher, is the same). In our example, if we replaced the constraint $quarter = 1997{-}Q3$ with $year = 1997-Q4$, retaining the rest of the query untouched, this creates a sibling query for the Date dimension. See [GVM15] for a detailed discussion of the general case.

Definition of a convenient variant for the sibling computation. A clear problem here is that if a value used in a selection atom has too many sibling values, this produces a large number of queries, even if we create siblings only for just this dimension. The problem generalizes if we take all selection atoms into consideration. To avoid this complexity we simplify sibling generation by (i) considering siblings only for the atoms ϕ_{α} and ϕ_{β}, and, (ii) by merging all sibling slices into one, by slightly adapting the aggregation level.

Given the original query q^{org}

$$q^{org} = \langle\, C^0,\, \phi_{\alpha} \wedge \phi_{\beta} \wedge \phi_{\square},\, [D_{\alpha}.L^{\gamma}_{\alpha},\, D_{\beta}.L^{\gamma}_{\beta}, M],\, agg(M^0)\, \rangle$$

we generate two sibling queries, each for one of the groupers as:

$$q^{s^{\alpha}} = \langle\, C^0,\, \phi^{\star}_{\alpha} \wedge \phi_{b} \wedge \phi_{\square},\, [D_{\alpha}.L^{\sigma}_{\alpha},\, D_{\beta}.L^{\gamma}_{\beta}, M],\, agg(M^0)\, \rangle,\, \phi^{\star}_{\alpha}\colon\, L^{\sigma}_{\alpha+1} = anc^{L^{\sigma}_{\alpha+1}}_{L^{\sigma}_{\alpha}}(v_{\alpha})$$

$$q^{s^{\beta}} = \langle\, C^0,\, \phi_{\alpha} \wedge \phi^{\star}_{\beta} \wedge \phi_{\square},\, [D_{\alpha}.L^{\gamma}_{\alpha},\, D_{\beta}.L^{\sigma}_{\beta}, M],\, agg(M^0)\, \rangle,\, \phi^{\star}_{\beta}\colon\, L^{\sigma}_{\beta+1} = anc^{L^{\sigma}_{\beta+1}}_{L^{\sigma}_{\beta}}(v_{\beta})$$

Intuitively, we perform the following two alterations to the original query:

- Instead of asking for the value of the filtering level of the dimension under moderation, we ask for its mother value. So, instead of a $state = CA$ filter, we introduce a filter $mama(state) = mama(CA)$, i.e., $country = USA$.
- We aggregate by the former filtering level. In this example, now that we have selected as the entire USA as the subspace to work with, we group by the former selector level, *state*, thus producing states as the grouper values of the result and effectively comparing all the sibling states in USA to one another (including our reference one, CA).

In our example:

$$q^{org} = \langle\ Sales,\ Date.Quarter\ =\ 1997Q3 \wedge Customer.State\ =\ 'CA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Month,\ Customer.Region,\ MORG],\ sum(StoreSales)\ \rangle$$

$$q^{s^{Date}} = \langle\ Sales,\ Date.Year\ =\ '1997'\ Customer.State\ =\ 'CA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Quarter,\ Customer.Region,\ MSDATE],\ sum(StoreSales)\ \rangle$$

$$q^{s^{Cust}} = \langle\ Sales,\ Date.Quarter\ =\ 1997Q3 \wedge Customer.Country\ =\ 'USA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Month,\ Customer.State,MSCUST],\ sum(StoreSales)\ \rangle$$

4.1.2 Drill-Down queries

Apart from contextualizing the result of the original query against its peers, we can also provide further details for the displayed data, by drilling into the dimension hierarchies of the produced results. Ideally, this requires drilling into each of the cells of the results of the original query – see [GVM15] on how this was traditionally done. However, this produces a significantly large number of queries to execute. We adopt a simple solution to this problem by drilling into each of the aggregator levels, by going down one level in their hierarchy. Thus, we produce two drill down queries, each drilling a level down into the detail of one of the grouper dimensions.

Given the original query q^{org}

$$q^{org} = \langle\ C^0,\ \phi,\ [D_\alpha.L^\gamma_\alpha,\ D_\beta.L^\gamma_\beta,M],\ agg(M^0)\ \rangle$$

we generate two drill-down queries, each for one of the groupers as:

$$q^{dd^\alpha} = \langle\ C^0,\ \phi,\ [D_\alpha.L^\gamma_{\alpha-1},\ D_\beta.L^\gamma_\beta,M^{dd^\alpha}],\ agg(M^0)\ \rangle$$

$$q^{dd^\beta} = \langle\ C^0,\ \phi,\ [D_\alpha.L^\gamma_\alpha,\ D_\beta.L^\gamma_{\beta-1},M^{dd^\beta}],\ agg(M^0)\ \rangle$$

In our example:

$$q^{org} = \langle\ Sales,\ Date.Quarter\ =\ 1997Q3 \wedge Customer.State\ =\ 'CA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Month,\ Customer.Region,\ MORG],\ sum(StoreSales)\ \rangle$$

$$q^{s^{Date}} = \langle\ Sales,\ Date.Year\ =\ '1997'\ Customer.State\ =\ 'CA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Quarter,\ Customer.Region,\ MSDATE],\ sum(StoreSales)\ \rangle$$

$$q^{s^{Cust}} = \langle\ Sales,\ Date.Quarter\ =\ 1997Q3 \wedge Customer.Country\ =\ 'USA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Month,\ Customer.State,MSCUST],\ sum(StoreSales)\ \rangle$$

$$q^{dd^{Date}} = \langle\ Sales,\ Date.Quarter\ =\ 1997Q3 \wedge Customer.State\ =\ 'CA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Day,\ Customer.Region,\ MDDDate],\ sum(StoreSales)\ \rangle$$

$$q^{dd^{Cust}} = \langle\ Sales,\ Date.Quarter\ =\ 1997Q3 \wedge Customer.State\ =\ 'CA' \wedge Promo.media\ =\ 'Daily\ Paper',\ [Date.Month,\ Customer.CustomerId,\ MDDCust],\ sum(StoreSales)\ \rangle$$

5 Multiple Query Optimization for Analyze Queries

So far, we have seen that the ANALYZE operator is actually a composition of 5 facilitator queries to the underlying database. In this section, we introduce a method that replaces the need of issuing five queries to the database with a merging of (a subset of) them, along with algorithms that exploit this merging by issuing a single query to (a) collect all the data that the 5 queries require, at an appropriate level of aggregation, and (b) post-process them, in memory, to produce the results of the 5 queries correctly. This is (a) costly, as the cost of multiple queries adds up the query overheads, and, (b) avoidable, as we will show in the sequel.

The method aims to gain in speed as (a) the query overheads are avoided, (b) multiple requests for the same data are reduced to a single fetch from the database (even, if intra-DBMS caching would have minimized this cost to a certain extent in the naive execution) and, (c) the fact that we are post-processing cells close to what would have been query results means that the amount of information we are post-processing is small – hence, we can do it efficiently in main memory.

5.1 Theoretical Background

We base our theoretical construction on the Cube Usability Theorem [Vas22, Vas23], which states that it is possible to derive the result of a cube query by filtering already accessed data by another cube query and re-aggregate them, if certain conditions are held.

For completeness, we repeat here the formulation of the theorem, and redirect the reader to the respective articles for the details of the proof and the supporting concepts.

Theorem 5.1 (Cube Usability ([Vas23, Vas22]))**.** Assume the following two queries:

$$q^n = \langle \mathbf{DS}^0,\ \phi^n,\ [L_1^n, \ldots, L_n^n, M_1, \ldots, M_m],\ [agg_1(M_1^0), \ldots, agg_m(M_m^0)]\ \rangle$$

and

$$q^b = \langle \mathbf{DS}^0,\ \phi^b,\ [L_1^b, \ldots, L_n^b, M_1, \ldots, M_m],\ [agg_1(M_1^0), \ldots, agg_m(M_m^0)] \rangle$$

The query q^b is *usable for computing*, or simply, *usable for* query q^n, if the following conditions hold:

i. both queries have exactly the same underlying detailed cube **DS**,

ii. both queries have exactly the same dimensions in their schema and the same aggregate measures $agg_i(M_i^0),\ i \in 1\ ..\ \ m$ (implying a 1:1 mapping between their measures), with all agg_i belonging to a set of known distributive functions.

iii. both queries have exactly one atom per dimension in their selection condition, of the form $D.L\ \in\ \{v_1, \ldots, v_k\}$ and selection conditions are conjunctions of such atoms,

iv. both queries have schemata that are perfectly rollable with respect to their selection conditions, which means that grouper levels are perfectly rollable with respect to the respective atom of their dimension,

v. all schema levels of query q^n are ancestors (i.e, equal or higher) of the respective levels of q^b, i.e., $D.L^b \preceq D.L^n$, for all dimensions D, and,

vi. for every atom of ϕ^n, say α^n, if (i) we obtain $\alpha^{n@L^b}$ (i.e., its detailed equivalent at the respective schema level of the previous query q^b, L^b) to which we simply refer as $\alpha^{n@b}$, and, (ii) compute its signature $\alpha^{n@b^+}$, then (iii) this signature is a subset of the grouper domain of the respective dimension at q^b (which involves the respective atom α^b and the grouper level L^b), i.e., $\alpha^{n@b^+} \subseteq gdom(\alpha^b, L^b)$.

In our case, we introduce the auxiliary, *all-encompassing query* q^A for the ANALYZE operator. The all-encompassing query q^A is characterized by: (0) the same basic cube and measure aggregation, (1) groupers involving the highest levels of aggregation that are low enough to answer any of the internal queries of the ANALYZE operator, and, (2) the broadest possible selection condition to encompass all the detailed tuples needed to answer any internal query. In the rest of this section we will first show that the all-encompassing query q^A can answer all the internal queries, and, second, we will accompany the feasibility result with two algorithms that actually compute the answer.

Theorem 5.2 (Multi-Query Usability)**.** Assume the query
analyze $= \langle\ C^0,\ \phi_\alpha \wedge \phi_\beta \wedge \phi_\square,\ [D_\alpha.L_\alpha^\gamma,\ D_\beta.L_\beta^\gamma, M],\ agg(M^0)\ \rangle$
producing five internal queries as prescribed in the definition of the ANALYZE operator. The following *all-encompassing query* q^A can answer all the internal queries of the ANALYZE operator.

$$q^A=\langle C^0,\ \phi^\star_\alpha \wedge \phi^\star_\beta \wedge \phi_\square,\ [L^\gamma_{\alpha-1}, L^\gamma_{\beta-1}, L^\gamma_\alpha, L^\gamma_\beta, L^\sigma_\alpha, L^\sigma_\beta,\ M^A],\ agg(M^0)\rangle,$$
$$\text{with } \phi^\star_\alpha\text{: } L^\sigma_{\alpha+1} = anc^{L^\sigma_{\alpha+1}}_{L^\sigma_\alpha}(v_\alpha) \text{ and } \phi^\star_\beta\text{: } L^\sigma_{\beta+1} = anc^{L^\sigma_{\beta+1}}_{L^\sigma_\beta}(v_\beta)$$

Proof. Some of the requirements of the Cube Usability Theorem are provided by construction of the operator and the all-encompassing query. We refer the reader to Table 1 as a clear demonstration of why the items in the following list hold. Specifically:

i. All queries are on the same detailed cube.

ii. All queries have the same dimensions, the same measure and distributive aggregate function.

iii. All queries have one atom per dimension of the form $D.L = v$ (*true* or equivalently, $D.ALL = all$ atoms are implied). In fact, the queries have non-trivial atoms for exactly the same dimensions, and even more, they can possibly differ only in the atoms of the grouper dimensions.

iv. All queries have selector levels higher than the grouper levels by construction. We have already assumed that for all dimensions D having $D.L^\gamma$ as a grouper level and $D.L^\phi$ as the level involved in the selection condition's atom for D, $D.L^\gamma \preceq D.L^\phi$, i.e., the selection condition is defined at a higher level than the grouping. Therefore, $D.L^\gamma_{i-1} \preceq D.L^\gamma_i \preceq D.L^\sigma_i$ by construction.

v. All queries have their grouper levels higher or equal to the ones of the all-encompassing one (again, remember that $D.L^\gamma_i \preceq D.L^\sigma_i$).

vi. Apart from the atoms for the grouper dimensions, all selection conditions in all queries are identical (i.e., they all share the same $\phi_\square$). The grouper dimensions' atoms also fulfill the constraint (vi) as we will show right away.

Therefore, all we need to show is the sixth constraint (iv), stating that once the all-encompassing query has been computed, it is possible to apply a selection condition to its result that will yield the same result as the original query. We start by showing that, given the result of the all-encompassing query which has been computed having applied $\phi^\star_\alpha$, it is attainable to produce the exact same result with having applied ϕ_α (the proof is identical for β).

Let us summarize the situation concerning the selection atom of dimension D_α.

- The "previous" query, which is the all-encompassing one, q^A, has a grouping level $D_\alpha.L^\gamma_{\alpha-1}$ and a selection condition $\phi^\star_\alpha$: $L^\sigma_{\alpha+1} = anc^{L^\sigma_{\alpha+1}}_{L^\sigma_\alpha}(v_\alpha)$. So practically, the produced grouper domain of q^A values for D_α are:
  v^g: s.t., $anc^{L^\sigma_{\alpha+1}}_{L^\gamma_{\alpha-1}}(v^g) = anc^{L^\sigma_{\alpha+1}}_{L^\sigma_\alpha}(v_\alpha)$,
  i.e., the children of v_α's mother (v_α's siblings) at the level $D_\alpha.L^\gamma_{\alpha-1}$.

	C^0	ϕ: $\phi_\alpha \wedge \phi_\beta \wedge \phi_\square$			Schema			agg
	$[i]$	$([iii][iv][vi])$			$([ii][iv][v])$			$[ii]$
q^A	C^0	$\phi^\star_\alpha$	$\phi^\star_\beta$	$\phi_\square$	$D_\alpha.L^\gamma_{\alpha-1}$	$D_\beta.L^\gamma_{\beta-1}$	M^A	$agg(M^0)$
q^{org}	C^0	ϕ_α	ϕ_β	$\phi_\square$	$D_\alpha.L^\gamma_\alpha$	$D_\beta.L^\gamma_\beta$	M^{org}	$agg(M^0)$
q^{dd^α}	C^0	ϕ_α	ϕ_β	$\phi_\square$	$D_\alpha.L^\gamma_{\alpha-1}$	$D_\beta.L^\gamma_\beta$	M^{dda}	$agg(M^0)$
q^{dd^β}	C^0	ϕ_α	ϕ_β	$\phi_\square$	$D_\alpha.L^\gamma_\alpha$	$D_\beta.L^\gamma_{\beta-1}$	M^{ddb}	$agg(M^0)$
q^{s^α}	C^0	$\phi^\star_\alpha$	ϕ_β	$\phi_\square$	$D_\alpha.L^\sigma_\alpha$	$D_\beta.L^\gamma_\beta$	M^{ssa}	$agg(M^0)$
q^{s^β}	C^0	ϕ_α	$\phi^\star_\beta$	$\phi_\square$	$D_\alpha.L^\gamma_\alpha$	$D_\beta.L^\sigma_\beta$	M^{ssb}	$agg(M^0)$

Table 1: Table for the proof of Multi-Query Usability of the all-encompassing query. The numbers in the header correspond to the id of the respective requirement in the Cube Usability Theorem.

Query	D_α: *Date*	D_β: *Customer*	$Grouper_\alpha$	$Grouper_\beta$
q^A	$Year = '1997'$	$Country = 'USA'$	$[Day$	$Customer]$
q^{org}	$Quarter = 1997Q3$	$State = 'CA'$	$[Month$	$Region]$
$q^{dd^{Date}}$	$Quarter = 1997Q3$	$State = 'CA'$	$[Day$	$Region]$
$q^{dd^{Cust}}$	$Quarter = 1997Q3$	$State = 'CA'$	$[Month$	$Customer]$
$q^{s^{Date}}$	$Year = '1997'$	$State = 'CA'$	$[Quarter$	$Region]$
$q^{s^{Cust}}$	$Quarter = 1997Q3$	$Country = 'USA'$	$[Month$	$State]$

Table 2: Reference example revisited for the sake of theoretical explanations

- Concerning D_α, all queries have either an atom ϕ_α: $L^\sigma_\alpha = v_\alpha$, or an atom $\phi^\star_\alpha$: $L^\sigma_{\alpha+1} = anc^{L^\sigma_{\alpha+1}}_{L^\sigma_\alpha}(v_\alpha)$.
- For the "new" queries with an atom $\phi^\star_\alpha$, there is nothing to be done: the all-encompassing one query has restricted the domain to exactly the needed values.
- The rest of the queries, must adapt the list of returned values from q^A to the ones they actually want, which are the ones satisfying ϕ_α. If this is the case, then an appropriate selection condition has to be applied to the results of q^A to select only the necessary values.
  The transformation of the ϕ_α atom to the level of q^A schema, $D_\alpha.L^\gamma_{\alpha-1}$ is:
  $D_\alpha.L^\gamma_{\alpha-1} \in desc^{L^\gamma_{\alpha-1}}_{L^\sigma_\alpha}(v_\alpha)$
  which produces the children of v_α, and, which, in turn is a clear subset of the grouper domain of q^A. So, if we apply this selection predicate to the results of q^A we restrict the set of D_α values to exactly the ones we need.

The same process is applied to ϕ_β and $\phi^\star_\beta$.

Then, the (vi) item of the checklist is proved, and, therefore, the entire theorem holds. □

Getting back to our example, Table 2 is showing only the parts of relevance from the query definitions. Observe that, concerning the *Date* dimension, the values produced by q^A are the days (grouper level is *Day*) of 1997 (selection atom is $Year = 1997$), i.e., $1997/01/01, 1997/01/02$, etc. Now, for all the queries who want the 3rd quarter of 1997, we need to reapply the filter for $Quarter = 1997Q3$, *but at the grouper level of* q^A*, i.e., Day*! Therefore, the filter applied to the results of q^A will be $Day \in desc^{Day}_{Quarter}(1997Q3)$. The fact that we filter at a high level and we group at a lower level guarantees us perfect rollability [Vas23, Vas22], which in turn allows us to reuse the results of the all encompassing query. Observe also that due to the fact that the groupers of q^A are lower or equal to the ones of all the other queries, we can further group the values of q^A to the higher levels of all the rest of the ANALYZE internal queries.

5.2 The Max Multi-Query Optimization Algorithm

The **MaxMQO Algorithm** (Algorithm 1) describes how to compute all the facilitator queries of the ANALYZE operator with single access to the underlying database. The algorithm takes as input the definition of the original query, which is sufficient to determine the entire set of facilitator queries and to produce their results as output. The steps of the algorithm is described below.

Step 1. First, the algorithm constructs five maps, one for each of the queries that determine the ANALYZE operator. Each of these maps will store the result of the respective query. The key of the map is the combination of grouper values (which are pretty much the coordinates of each result cell), and the value is the aggregate measure that pertains to these coordinates.

Step 2. Second, the algorithm constructs and executes the all-encompassing auxiliary query q^A that will serve as the basis to compute all the other query results.

Step 3. Once the tuples of q^A have been computed, we need to distribute them to the facilitator queries of the operator. We visit each tuple at a time. All tuples will end up in the siblings, as the selection condition of q^A is exactly that of the siblings. Recall that the selection condition of the siblings is broader than the one of the original query and the drill-downs (which is identical to the original one). However, some of the resulting tuples also pertain to the original and the drill-down queries, and so, they have to be pushed towards the respective maps. In the end, the hash-maps contain the results of the auxiliary queries.

For example, assume that the user issues the following *ANALYZE* query:

```
ANALYZE sum(store_sales) FROM sales
FOR state = 'CA' AND quarter = '2025-Q4'
```

Algorithm 1: Algorithm MaxMQO

Input: Original query $q^{org} = \langle\ C^0,\ \phi,\ [D_\alpha.L^\gamma_\alpha,\ D_\beta.L^\gamma_\beta, M],\ agg(M^0)\ \rangle$,
phi: $\phi_\alpha \wedge \phi_\beta \wedge \phi_\square$, $phi_i : D_i.L^\sigma_i = v_i,\ i : \alpha\ or\ \beta$

Output: Updated results for q^{org}, q^{s^A}, q^{s^B}, q^{dd^A}, q^{dd^B}

begin
- Let $H^{org}, H^{s^A}, H^{s^B}, H^{dd^A}, H^{dd^B}$ be maps of the form $H : \langle\ dom(L^\gamma_1), dom(L^\gamma_2)\ \rangle \rightarrow dom(M^0)$;
- Let $aggF^\star = adapter(aggF)$;
- `/* Construct and execute the MQO query */`
- Let $\phi^\star_\alpha$: $L^\sigma_{\alpha+1} = anc^{L^\sigma_{\alpha+1}}_{L^\sigma_\alpha}(v_\alpha)$ and $\phi^\star_\beta$: $L^\sigma_{\beta+1} = anc^{L^\sigma_{\beta+1}}_{L^\sigma_\beta}(v_\beta)$;
- Let $q^A = \langle\ C^0,\ \phi^\star_\alpha\ \wedge\ \phi^\star_\beta\ \wedge\ \phi_\square,\ [L^\gamma_{\alpha-1}, L^\gamma_{\beta-1}, L^\gamma_\alpha, L^\gamma_\beta, L^\sigma_\alpha, L^\sigma_\beta,\ M^A],\ agg(M^0)\ \rangle$;
- $q^A.cells = q^A.execute()$;
- **foreach** *tuple t in the result set* $q^A.cells$ **do**
  - Assume $t = \langle v^\gamma_{\alpha-1}, v^\gamma_{\beta-1}, v^\gamma_\alpha, v^\gamma_\beta, v^\sigma_\alpha, v^\sigma_\beta, m\rangle$;
  - `/* if t satisfies` $\phi_\alpha \wedge \phi_\beta$`: update org & dd's */`
  - **if** $\sigma_{\phi_\alpha \wedge \phi_\beta}(t)$ **then**
    - $updateMap(H^{org}, \langle\langle v^\gamma_\alpha, v^\gamma_\beta\rangle, m\rangle, agg)$;
    - $updateMap(H^{dd^A}, \langle\langle v^\gamma_{\alpha-1}, v^\gamma_\beta\rangle, m\rangle, agg)$;
    - $updateMap(H^{dd^B}, \langle\langle v^\gamma_\alpha, v^\gamma_{\beta-1}\rangle, m\rangle, agg)$;
  - **end**
  - **if** $\sigma_{\phi_\beta}(t)$ **then**
    - $updateMap(H^{s^A}, \langle\langle v^\sigma_\alpha, v^\gamma_\beta\rangle, m\rangle, agg)$;
  - **end**
  - **if** $\sigma_{\phi_\alpha}(t)$ **then**
    - $updateMap(H^{s^B}, \langle\langle v^\gamma_\alpha, v^\sigma_\beta\rangle, m\rangle, agg)$;
  - **end**
- **end**
- $q^{org}.cells = H^{org}$; $q^{dd^A}.cells = H^{dd^A}, q^{dd^B}.cells = H^{dd^B}$;
- $q^{s^A}.cells = H^{s^A}$; $q^{s^B}.cells = H^{s^B}$;
- **return** $\langle q^{org}.cells, q^{s^A}.cells, q^{s^B}.cells, q^{dd^A}.cells, q^{dd^B}.cells\rangle$;

end

Function `updateMap`($H : map,\ \langle\langle g_1, g_2\rangle, m\rangle :$ $Tuple\langle Pair\langle grouper, grouper\rangle, measure\rangle,\ aggF^\star : agg.\ Func.$`):`
- **if** $H.containsKey(\langle g_1, g_2\rangle)$ **then**
  - $curValue = H.get(\langle g_1, g_2\rangle)$;
  - $H.put(\langle g_1, g_2\rangle, aggF^\star(currValue, m))$;
- **end**
- **else**
  - $H.put(\langle g_1, g_2\rangle, m)$;
- **end**

GROUP BY state,quarter

This query is regarded as the original query q^{org}. Step 1 constructs five maps for each facilitator query result $H^{org}, H^{s^A}, H^{s^B}, H^{dd^A}, H^{dd^B}$. In step 2, the all-encompassing query q^A is constructed by replacing the selection condition of q^{org} with their parent levels and values. In addition, four more groupers are added in the current set, which contains the child and parent levels of the q^{org} groupers. The all-encompassing query is:

ANALYZE sum(store_sales) FROM sales
FOR country = 'USA' AND year = '2025'
GROUP BY city, month, state, quarter, country, year

Then, this all-encompassing query is translated to SQL and executed. The result contains the result tuples of all facilitator queries. In Step 3, each result tuple is visited and distributed to a result bucket $H^{org}, H^{s^A}, H^{s^B}, H^{dd^A}, H^{dd^B}$ that was created in Step 1 if it is part of the result of the respective facilitator query $(q^{org}, q^{s^A}, q^{s^B}, q^{dd^A}, q^{dd^B})$, with respect to the cube usability theorem [Vas22, Vas23].

Observe also how the tuples are integrated in any of the maps. The method *updateMap()* implements this handling: (a) If the coordinates of the tuple do not already exist in the map, we have to insert the incoming tuple in the map, as the first of its group. (b) Otherwise, if the coordinates already exist, we need to update the aggregate measure. Bear in mind that we cannot directly apply the *agg* function: whereas the aggregate function is directly applicable for the case of *min*, *max* and *sum*, for the case of *count*, we simply have to increase the counter by one. To facilitate this, we introduce the *adapter* aggregate function that takes care of this problem by adapting the actual combination of the current and the new value accordingly.

5.3 Mid - Multi Query Optimization Algorithm for the Analyze Operator

The Max Multi-Query Optimization Algorithm utilizes a single query on the underlying database to retrieve the result of an *ANALYZE* query. However, the selection conditions of the all-encompassing auxiliary query q^A explore a vast super-set of tuples that contain the query result for each of the five queries, plus tuples that are not part of any query result, thus they are not aggregated. The spare tuples add performance latency, as they have to be processed by Max MQO to determine whether a tuple belongs to one of the five query result maps.

To reduce the vastness of the explored data space, we can reduce the number of spare tuples returned by q^A. Observe that the selection conditions of the original query and the drill-down queries are the same. We take advantage of that property to construct a single query q^{MID} that combines the original and drill-down facilitator queries, without touching the siblings.

The **Mid MQO Algorithm** (Algorithm 2) launches the two sibling queries along with the merged q^{MID} query. Since the latter pertains to three facilitator queries, we group the tuples on their original and ancestor levels to distribute the result tuples. In that way, we get the tuples that contribute to the original and drill-down queries result in a single access. The rest of the processing is the same as with Max MQO. Compared to q^A, q^{MID} returns less detailed tuples and applies less groupers to them, thus reducing the result size.

Algorithm 2 presents the implementation of the *Mid Multi-Query Optimization*. First, the algorithm constructs three maps, one for the original query and one for each drill-down query that determine the *ANALYZE* operator. Each of these maps will store the result of its respective query. The key of the map is the combination of grouper values and the value is the aggregate measure that pertains to these coordinates (i.e., <grouper1,grouper$_2$> -> m). The resulting tuples are pushed into the respective buckets via *updateMap()* as described in section 5.2.

Then, the sibling queries q^{sA}, q^{sB} are constructed and executed separately to return the exact sibling result tuples. In this way, we avoid the presence of spare tuples by applying selection conditions that involve only parent values and result in exploring a large part of the fact table. Finally, the result tuples of the three queries are packaged to create the ANALYZE query result.

For example, assume that the user issues the following *ANALYZE* query:

```
ANALYZE sum(store_sales) FROM sales
FOR state = 'CA' AND quarter = '2025-Q4'
GROUP BY state,quarter
```

This query is regarded as the original query q^{org}. The Mid-MQO algorithm constructs three maps for each facilitator query result $H^{org}, H^{dd^A}, H^{dd^B}$. Then, the merged query q^{MID} is constructed by maintaining the selection conditions of q^{org} and by adding, two more groupers in the current set. The groupers of the merged query contain the child levels of the q^{org} groupers. The merged query is:

```
ANALYZE sum(store_sales) FROM sales
FOR state = 'CA' AND quarter = '2025-Q4'
GROUP BY city, month, state, quarter
```

Then, the sibling queries q^{s^A}, q^{s^B} are constructed with respect to the syntax discussed in Section 4. Then, the merged query q^{MID} and the sibling queries q^{s^A}, q^{s^B} are translated to SQL and executed. The result of sibling queries contains the exact sibling result tuples. The merged query contains the result tuples of the original and drill-down queries. Similarly, as described in the Max-MQO algorithm, each tuple is visited and distributed to a result bucket $H^{org}, H^{dd^A}, H^{dd^B}$ if it is part of the respective facilitator query ($q^{org}, q^{dd^A}, q^{dd^B}$) with respect to the cube usability theorem [Vas22, Vas23].

Algorithm 2: Algorithm Mid Multi-Query Optimization

Input: Original query $q^{org} = \langle C^0, \phi, [D_\alpha.L^\gamma_\alpha, D_\beta.L^\gamma_\beta, M], agg(M^0)\rangle$,
$\phi = \phi_\alpha \wedge \phi_\beta \wedge \phi_\square, \quad \phi_i : D_i.L^\sigma_i = v_i,\ i \in \{\alpha, \beta\}$

Output: Updated results for $q^{org}, q^{s^A}, q^{s^B}, q^{dd^A}, q^{dd^B}$

begin
  Let $H^{org}, H^{s^A}, H^{s^B}, H^{dd^A}, H^{dd^B}$ be maps of the form
    $H : \langle dom(L^\gamma_1), dom(L^\gamma_2)\rangle \rightarrow dom(M^0)$;
  Let $aggF^\star = adapter(aggF)$;
  `/* Construct and execute Mid MQO queries */`
  Let $\phi^*_\alpha : L^\sigma_{\alpha+1} = anc_{L^\sigma_{\alpha+1}L^\sigma_\alpha}(v_\alpha)$;
  Let $\phi^*_\beta : L^\sigma_{\beta+1} = anc_{L^\sigma_{\beta+1}L^\sigma_\beta}(v_\beta)$;
  Let
    $q^{org\&dd} = \langle C^0, \phi_\alpha \wedge \phi_\beta, [L^\gamma_{\alpha-1}, L^\gamma_{\beta-1}, L^\gamma_\alpha, L^\gamma_\beta, M^{org\&dd}], agg(M^0)\rangle$;
  Let $q^{s^A} = \langle C^0, \phi^*_\alpha \wedge \phi_\beta, [L^\gamma_{\alpha+1}, L^\gamma_\beta, M^{s^A}], agg(M^0)\rangle$;
  Let $q^{s^B} = \langle C^0, \phi_\alpha \wedge \phi^*_\beta, [L^\gamma_\alpha, L^\gamma_{\beta+1}, M^{s^B}], agg(M^0)\rangle$;
  $q^{org\&dd}.cells \leftarrow q^{org\&dd}.execute()$;
  $q^{s^A}.cells \leftarrow q^{s^A}.execute()$;
  $q^{s^B}.cells \leftarrow q^{s^B}.execute()$;
  foreach *tuple* $t \in q^{org\&dd}.cells$ **do**
    Assume $t = \langle v^\gamma_{\alpha-1}, v^\gamma_{\beta-1}, v^\gamma_\alpha, v^\gamma_\beta, m\rangle$;
    if $\sigma_{\phi_\alpha \wedge \phi_\beta}(t)$ **then**
      $updateMap(H^{org}, \langle\langle v^\gamma_\alpha, v^\gamma_\beta\rangle, m\rangle, agg)$;
      $updateMap(H^{dd^A}, \langle\langle v^\gamma_{\alpha-1}, v^\gamma_\beta\rangle, m\rangle, agg)$;
      $updateMap(H^{dd^B}, \langle\langle v^\gamma_\alpha, v^\gamma_{\beta-1}\rangle, m\rangle, agg)$;
    end
  end
  $q^{org}.cells \leftarrow H^{org}$;
  $q^{dd^A}.cells \leftarrow H^{dd^A}$, $q^{dd^B}.cells \leftarrow H^{dd^B}$;
  return $\langle q^{org}.cells, q^{s^A}.cells, q^{s^B}.cells, q^{dd^A}.cells, q^{dd^B}.cells\rangle$;
end
Function `updateMap(`$H : map, \langle\langle g_1, g_2\rangle, m\rangle$:
$Tuple\langle Pair\langle grouper, grouper\rangle, measure\rangle, aggF^\star$`):`
  if $H.containsKey(\langle g_1, g_2\rangle)$ **then**
    $curValue \leftarrow H.get(\langle g_1, g_2\rangle)$;
    $H.put(\langle g_1, g_2\rangle, aggF^\star(curValue, m))$;
  end
  else
    $H.put(\langle g_1, g_2\rangle, m)$;
  end

5.4 A note on the design choices of our algorithms

Before proceeding to present the experimental evaluation of these algorithms, it is worthwhile to stop for a quick comment on the fundamental design choices that guide them. We introduce an operator that ultimately invokes several database queries (even if we produce an algorithm to merge them into one). Also, the optimizations that we introduce are (a) DBMS-external, and therefore, DBMS-agnostic, and, (b) performed automatically at the syntactic level of query rewriting, rather than exploring alternative multi-query optimizations.

One important lesson to be learned here is that we can introduce a level of abstraction above the direct database querying that resembles more naturally the thinking of the analyst (much like Roll-Ups and Drill-Downs were introduced for OLAP). These high level operators, internally, can involve several facilitator queries, and therefore incur execution costs, and, at the same time, provide optimization opportunities.

At the same time, observe that all our algorithms *are using* cube *queries, rather than* database *queries, as the granule of work*. As a result, the rewritings of the original query that these algorithm perform is performed by exploiting the combination of filters and groupers at the hierarchical multidimensional space and do *not* involve the exploration of alternative plans inside the DBMS.

Overall, the entire design offers another important lesson to be learned: *working at the cube query level,* outside *the DBMS can incur significant performance gains*.

Of course, integrating such concepts inside the DBMS is also worth exploring, although outside the scope of the current paper. If no optimizations were introduced, either inside or outside the DBMS, the execution of ANALYZE ends up being the execution of the Min-MQO algorithm, i.e, the batch execution of five database queries, one after the other. Apart from optimizing this outside the DBMS, we can also consider the possibility of expanding the DBMS with the operator, in which case, the optimizer of the DBMS could also be expanded too. This involves both the MQO possibilities listed here, and also, other opportunities –for example, deciding the join order in this context is worth exploring. Observe that, in contrast to traditional MQO algorithms, exactly because of the characteristics of the data space, we *do not seek* to find rewritings, we automatically (and therefore efficiently) perform them, by exploiting the hierarchies.

6 Experiments

In this section, we experimentally evaluate the execution time of the proposed algorithms under varying configurations of data and query workloads. In all of our experiments, we use the Delian Cubes system [Del25], which is a cube query answering engine, within which we have incorporated our algorithms. All algorithms have been implemented in Java, as part of the Delian Cubes system. We have used MySQL 8.0.34 as the underlying database. For our evaluation, the methods run on a Windows 11 2.50 GHz 14-core processor system with

32GB main memory and 1TB SSD. All material is found at: `https://github.com/DAINTINESS-Group/DelianCubeEngine`.

6.1 Experimental Setup

Experimental Goal and Evaluation Metrics. The main goal of our evaluation is to evaluate the efficiency of the proposed algorithms to execute the ANALYZE operator. Therefore, for all antagonists, we measure the *Total Execution Time* needed to fully compute the answer to an ANALYZE query, as well as its breakdown to different facilitators within each algorithm. *Total Query Execution Time* is the sum of the time taken by different parts of each algorithm, and specifically: (i) *Parsing Time* for the string input expression of the ANALYZE query, (ii) *Construction Time* for the facilitator queries, (iii) *Facilitator Execution Time* for executing the facilitator queries, (iv) *Post-processing Time* of the results of the facilitator queries and population of the ANALYZE query result correctly.

Competitor Algorithms. In our experiments, we compare the three different algorithms to implement the *ANALYZE* operator on *Total Execution Time* and its breakdown. For the convenience of the reader, we briefly summarize these three methods in both Table 3 and the following short summary. All algorithms receive as input an ANALYZE query with two groupers, a distributive aggregate function, and at least two selection conditions. Recall that the semantics of the operator entail the execution of 5 internal *facilitator* cube queries, namely the original one q^{org}, two sibling q^{s^A}, q^{s^B} and two drilling cube queries q^{dd^A}, q^{dd^B}.

- *Min-MQO* is the algorithm that performs the least merging of the operator's underlying cube queries (thus, "Minimum Multi-Query Optimization"). As already mentioned, the algorithm constructs the five facilitator queries as described by the operator's semantics. Practically, this is the plain simple execution of the operator without any optimization and as such, it also avoids having to perform any post-processing to the results obtained from the executed queries.

- *Max-MQO* is the algorithm that performs the maximum merging of the facilitator queries into a single facilitator query (thus, "Maximum Multi-Query Optimization") with four extra groupers to cover the *siblings* and *drill-down* results, substituted selection conditions (input atoms are substituted by their parent levels and parent values). The results are post-processed and the tuples are distributed to the correct placeholder with respect to the operator semantics.

- *Mid-MQO* is the algorithm that strikes a balance in the middle of the two extremes of full -and no - merging of the facilitator queries into one.

Specifically, the algorithm constructs three facilitator queries: (i) two *sibling* queries, one for each grouping dimension, and, (ii) a merged *original-n-drill-down* multi-query which is a combination of the *original* and *drill-down* queries, which exploits the fact that these queries have the exact same selection conditions and thus search the same data space of tuples in the fact table. The resulting tuples are post-processed and distributed to the correct placeholder as in the previous case.

Name	**# Executable Queries**	**Post-Processing**
Min-MQO	5	No
Mid-MQO	3	Yes
Max-MQO	1	Yes

Table 3: Details of Methods

Datasets. We have used the datasets listed in Table 4 to perform our evaluation. Specifically, these datasets are: (i) *Northwind* [Mic23], which is a synthetic dataset that represents the sales and the operations of a food import/export company, (ii) *Foodmart*, which is a cubefied version of *Foodmart* [Hyd25], which contains synthetic data regarding the sales activity of a retail supermarket, (iii) *pkdd99+*, which is an upscaled version of *pkdd99* [ZYO99], a financial dataset that contains data about loans and transactions, for which we have created a fact table that contains 100 million entries, and, (iv) *TPC-DS* [TPC24], which is an industry standard benchmark designed to evaluate the performance of analytical platforms. We have tested the scalability of our algorithms over three versions of *TPC-DS* with scale factor 1, 3.5, and, 35. As Delian Cubes operates over data cubes, we do not directly utilize the raw schema of the datasets, but rather adapt it to a cubefied, clean, star-schema version, in order to support hierarchies and cube queries.

Name	**Scale Factor**	**# Dim. Tables**	**# Facts**
Northwind	1	4	2K
Foodmart	1	5	288K
pkdd99+	166000	3	100M
TPC-DS	1,3.5,35	6	2M,10M,100M

Table 4: Dataset Basic Characteristics

Query Workloads. The query workloads cover multiple cases of an ANALYZE query. The characteristics of an ANALYZE query are (i) the *selectivity ratio*, (ii) the *level of filter/grouper values*, and (iii) *the number of filters*. The selectivity ratio refers to the percentage of tuples in the fact table that are involved in the computation of the final query result. The level of filter/grouper

QueryID	Filter Level	Grouper Level	#Filters	Selectivity Ratio			
				q^{org}	q^{sib_1}	q^{sib_2}	q^A
1	Mid-High	Mid-High	2	1%	8%	3%	20%
2	Mid-High	Mid-High	2	1%	4%	4%	9%
3	Low-High	Low-High	2	1.5%	8%	6%	20%
4	Mid-High	Mid-High	2	2%	4%	8%	20%
5	Mid-High	Mid-High	2	3%	8%	9%	20%
6	Mid-High	Mid-High	2	3.5%	8%	3%	20%
7	High-High	High-High	2	6%	30%	20%	100%
8	High-High	High-High	2	6.5%	30%	20%	100%
9	High-High	High-High	2	8%	37%	20%	100%
10	High-High	High-High	2	8%	37%	20%	100%

Table 5: TPC-DS Time Workload Characteristics

QueryID	Filter Level	Grouper Level	#Filters	Selectivity Ratio			
				q^{org}	q^{sib_1}	q^{sib_2}	q^A
1	Low-Low	Low-Low	2	0.001%	0.001%	0.35%	1%
2	Mid-Low	Mid-Low	2	0.002%	0.01%	0.35%	2%
3	High-Low	High-Low	2	0.01%	0.03%	1%	10%
4	Low-High	Low-High	2	0.2%	0.5%	2%	5%
5	Low-High	Low-High	2	0.35%	0.5%	2%	9%
6	Mid-High	Mid-High	2	0.5%	1%	3.5%	20%
7	Mid-High	Mid-High	2	0.9%	2%	6%	20%
8	Mid-High	Mid-High	2	1%	10%	20%	20%
9	High-High	High-High	2	2%	2%	9%	100%
10	High-High	High-High	2	2%	9%	10%	100%

Table 6: TPC-DS Item Workload Characteristics

QueryID	Filter Level	Grouper Level	#Filters	Selectivity Ratio			
				q^{org}	q^{sib_1}	q^{sib_2}	q^A
1	Low-Low	Low-Low	2	0.01%	0.01%	0.2%	0.2%
2	Mid-Low	Mid-Low	2	0.01%	0.25%	0.15%	3%
3	Mid-Low	Mid-Low	2	0.2%	0.2%	2%	2%
4	High-Low	High-Low	2	0.6%	3%	4%	20%
5	High-High	High-High	2	1.4%	17%	11%	100%
6	High-High	High-High	2	1.7%	17%	9%	100%
7	High-Low	High-Low	2	2%	2%	13%	13%
8	Mid-High	Mid-High	2	3%	17%	18%	100%
9	High-High	High-High	2	3%	17%	18%	100%
10	High-High	High-High	2	3%	17%	18%	100%

Table 7: pkdd99+ Workload Characteristics

QueryID	Filter Level	Grouper Level	#Filters	Selectivity Ratio			
				q^{org}	q^{sib_1}	q^{sib_2}	q^A
1	Low-Low	Low-Low	2	0.001%	1%	0.01%	3%
2	Low-Low	Mid-Mid	2	0.001%	1%	0.01%	3%
3	Low-Low	High-High	2	0.001%	1%	0.01%	3%
4	Mid-Mid	Low-Low	2	3%	9%	9%	30%
5	Mid-Mid	Mid-Mid	2	3%	9%	9%	30%
6	Mid-Mid	High-High	2	3%	9%	9%	30%
7	High-Mid	Low-Low	2	9%	30%	19%	67%
8	High-High	Low-Low	2	30%	30%	67%	100%
9	High-High	Mid-Mid	2	30%	30%	67%	100%
10	High-High	High-High	2	30%	30%	67%	100%

Table 8: Foodmart Workload Characteristics

QueryID	Filter Level	Grouper Level	#Filters	Selectivity Ratio			
				q^{org}	q^{sib_1}	q^{sib_2}	q^A
1	Low-Low	Low-Low	2	0.5%	6%	1%	13%
2	Low-Low	Mid-Mid	2	0.5%	6%	1%	13%
3	Low-Low	High-High	2	0.5%	6%	1%	13%
4	Mid-Mid	Low-Low	2	1%	13%	1%	17%
5	Mid-Mid	Mid-Mid	2	1%	13%	1%	17%
6	High-Mid	Low-Low	2	6%	28%	13%	74%
7	High-Mid	High-High	2	6%	28%	13%	74%
8	High-High	Low-Low	2	13%	74%	17%	100%
9	High-High	Mid-Mid	2	13%	74%	17%	100%
10	High-High	High-High	2	13%	74%	17%	100%

Table 9: Northwind Workload Characteristics

values is a relative characterization of the height of the filter/grouper level in the hierarchy of its dimension. Tables 5, 6, 7, 8, 9 summarize the workload characteristics of each dataset. To assess the operator with different selectivities over the fact tables, for each data set, we use a workload of 10 queries, where we (a) vary with the various levels of filters/groupers, and (b) employ one large and one small dimension table. For the TPC-DS dataset, and in order to also evaluate the effect of dimension table size, we use two query workloads, both involving the large *Date* dimension: (i) a workload using the large *Time* dimension table and (b) a workload using the small *Item* dimension table. In all workloads, as the QueryID increases, the selectivity of q^{org} also increases in all data sets. The two workloads differ only on the selectivity ratio due to the difference in the dimension table size. The *Time* workload involves two relatively large dimension tables and the *Item* workload one large and one small. Both workloads involve the large *Date* dimension. For the rest of the datasets, we employed a single workload of 10 queries with various levels of filters/groupers

that involve one large and one small dimension table. To ensure that the result size of each ANALYZE query is relatively small in order to be comprehensible and manageable, we set the grouper levels of each ANALYZE query at the highest level possible with respect to the constraints discussed in Section 3.

6.2 Effect of Query Characteristics on the Performance of the Algorithms

In this subsection, we discuss the effect of each of the query characteristics on the overall performance of the antagonist algorithms. In all the presented Figures, each bar represents the total execution time, with Min-MQO shown as a dark blue bar, Mid-MQO as a blue bar, Max-MQO as a light blue bar, and the timeout limit as a red dotted line. In the experiments that follow, we want to assess the effect of each of these characteristics on the performance of the different algorithms. To this end, we define a reference query and modify the values of the assessed characteristic while keeping the rest of the characteristics fixed. We used the *TPC-DS* dataset that contains 100 million facts. The reference values for the query characteristics are: 20% selectivity ratio, mid grouper levels, and 2 filters per query. The values of each tested query characteristic are prescribed in the respective subsection.

6.2.1 Effect of Selectivity Ratio

In this experiment, we evaluate how the selectivity ratio of the ANALYZE query affects its execution time for the three antagonist algorithms. Figure 2 presents the effect of an ANALYZE query selectivity ratio on the query execution time for all methods. The selectivity ratios that we utilized for our experiment are 10%/50%/90% of the number of fact table tuples. For all antagonists, the query execution time scales, due to the increasing number of tuples processed.

Observe that in all these setups, Mid-MQO consistently outperforms the antagonist methods. For low selectivity, Min-MQO and Max-MQO are close because the number of tuples being processed is small. Both Min-MQO and Mid-MQO scale similarly with selectivity; on the other hand, Max-MQO quickly deteriorates as selectivity increases (recall that internally, Max-MQO has the largest interim result set to process).

6.2.2 Effect of Number of Atomic Filters

In this experiment, we evaluate how the number of atomic filters in the selection condition of an ANALYZE query affects its execution time for the three antagonist algorithms. Figure 3 shows the effect of the number of filters per query on the query execution time for all methods. The number of atomic filters utilized for our evaluation is 2/3/4/5. It is important to note that the presence of join operations and the number of filters are directly correlated: more filters require that more dimension tables be joined. Naturally, the number of filters also directly affects the selectivity ratio, as the number of atoms of the selection

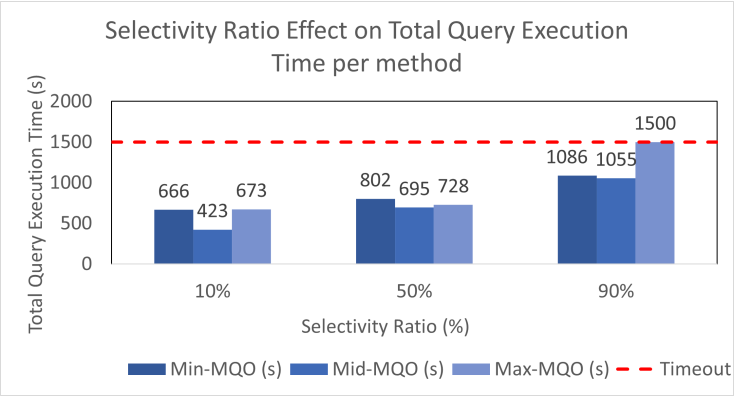


Figure 2: Selectivity Ratio effect on Total Querying Execution Time. *Max-MQO* did not complete its execution on the 90% selectivity ratio case.

condition is increasing. This is due to the nature of the query and signifies that, although more joins are added, the effect of join processing is annulated by the number of tuples involved in the processing of the facilitator queries. As we observe Figure 3, a few facts become evident:

- As already mentioned, the decrease of the number of involved tuples directly shrinks the execution time for all algorithms.
- Max-MQO gains performance much faster than the other algorithms as the data volume decreases due to the increase of the selectivity ratio, utilizing the single access to the database. Ultimately, in very low numbers of detailed fact tuples involved, Max-MQO wins.
- Min-MQO and Mid-MQO are not affected by the shift in the number of filters in a fashion similar to that of Max-MQO. Both of these methods are accessing the database multiple times and perform multiple join operations on multiple queries due to the extra filters. As the number of tuples is retained at high levels, this need for extra joins reduces the benefits of the tuples involved compared to Max-MQO, which accesses the database once. In all occasions but one, Min-MQO is the worst-performing algorithm.

6.2.3 Other considerations

There are a couple of results that we briefly summarize here.

- *Effect of the level of aggregation due to the groupers.* We have varied the "height" of the grouper levels in their respective hierarchy. To this end, we classify the "height" of a level as *low* (when it is the lowest level of the hierarchy), *high* (the topmost ALL, as well as its immediate child), and *mid* otherwise. Our experiments have demonstrated that the level of aggregation of groupers has no effect on Total Execution Time (Figure 4).

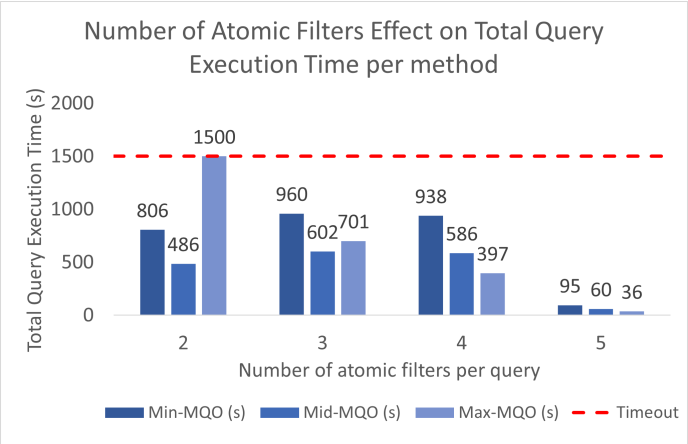


Figure 3: Number of Atomic Filters Effect. *Max-MQO* did not complete its execution for 2 atomic filters.

- *Time breakdown.* We have analyzed the internal behavior of the alternative algorithms, to detect possible points of delay. To this end, we have split the query execution process to four stages, specifically (a) parsing, (b) construction of auxiliary facilitator queries, (c) execution time of all facilitator queries, and, (d) post-processing of the intermediate results, to construct the final query answer (no need for this in the case of Min-MQO). To cover the entire spectrum of fact table coverage by the original query, we varied the selectivity ratio of the original query to 10%, 50%, 90% of the fact table tuples. The breakdown of Total Execution Time into the aforementioned four stages, reveals that in all cases, the Facilitator Execution Time takes approximately 99% of Total Query Execution Time, making it the nexus of any optimization effort (Figure 5).

6.3 Effect of Scaling on Performance

To assess the effect of data size on the performance of the algorithms, we had to subject the system to a workload of queries, each with different characteristics. We utilize the TPC-DS *Item* and *Time* workloads listed in Tables 5 and 6 and include a set of ANALYZE queries with various combinations of characteristics.

Figures 6a, 6b, 6c present the effect of scaling data size on the query execution time across all methods for the *Item* workload. Min-MQO shown as dark blue , Mid-MQO as blue, and Max-MQO as light blue. The number of fact table entries of the same dataset utilized for our evaluation is 2/10/100 million. It is evident that as the data size scales, the query execution time of all algorithms is increasing. Observe that Max-MQO's performance drops significantly as the number of fact table entries increases and the performance gap between the other antagonists is increasing. Min-MQO's performance closely follows that of

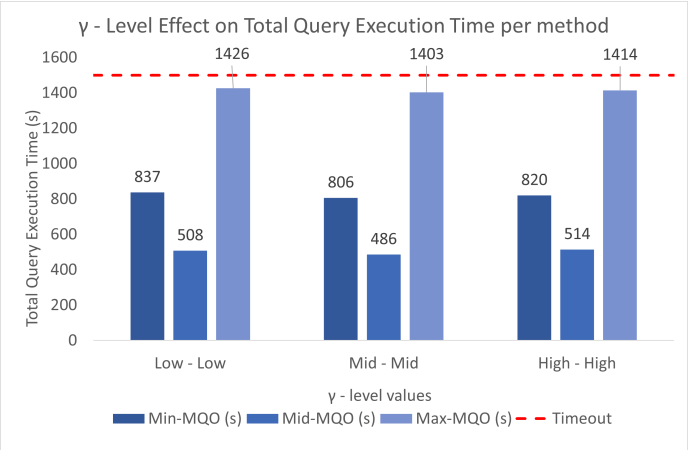


Figure 4: Grouper Effect on Total Execution Time

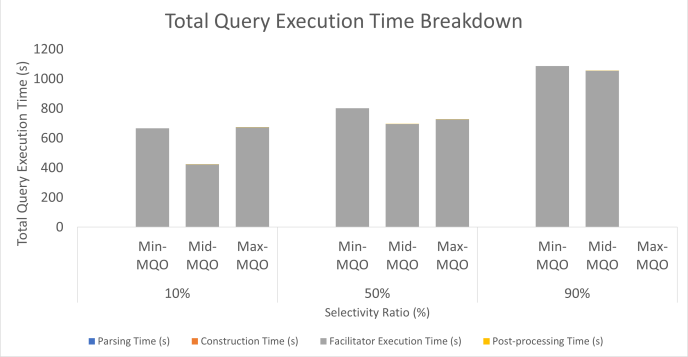
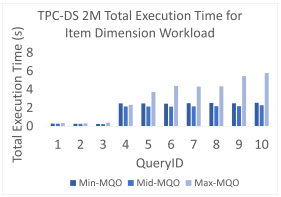


Figure 5: Total Execution Time Breakdown

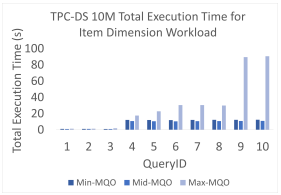
Mid-MQO, but Mid-MQO is always faster. Figures 6g, 6h, 6i present the effect of scaling data size on the query execution time for all methods for the *Time* workload. It is evident that as the data size scales, the query execution time of all algorithms is increasing. In this case, the dimension tables are large, and the sibling facilitator queries do not dominate the execution time of the ANALYZE query. We observe that Max-MQO's performance scales better with the data size, in comparison to its antagonists, and is the fastest algorithm in almost every query. Min-MQO's performance closely follows that of Mid-MQO, but Mid-MQO is always faster.

6.4 Performance Evaluation

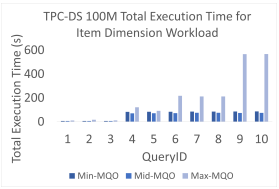
To assess the performance of the antagonist methods, we evaluate each algorithm's performance on various datasets and data sizes. Figure 6 presents the



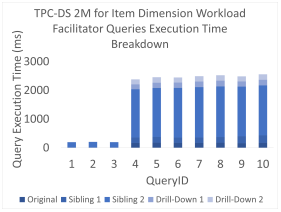
(a) TPC-DS 2M *Item* Workload



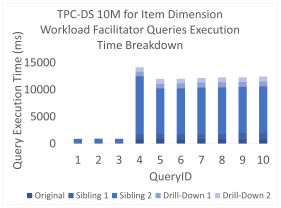
(b) TPC-DS 10M *Item* Workload



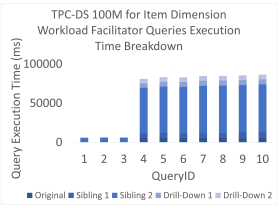
(c) TPC-DS 100M *Item* Workload



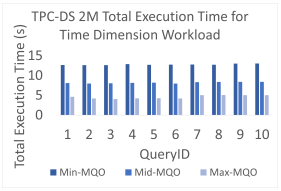
(d) TPC-DS 2M *Item* Workload Facilitator Queries Breakdown



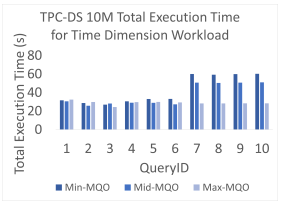
(e) TPC-DS 10M *Item* Workload Facilitator Queries Breakdown



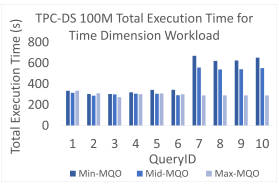
(f) TPC-DS 100M *Item* Workload Facilitator Queries Breakdown



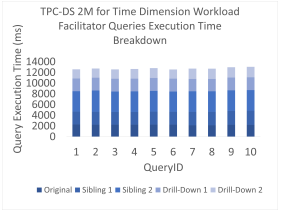
(g) TPC-DS 2M *Time* Workload



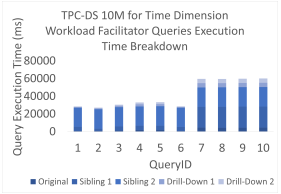
(h) TPC-DS 10M *Time* Workload



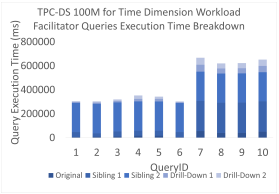
(i) TPC-DS 100M *Time* Workload



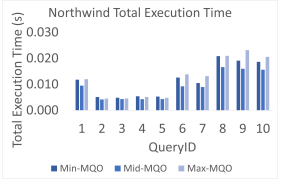
(j) TPC-DS 2M *Time* Workload Facilitator Queries Breakdown



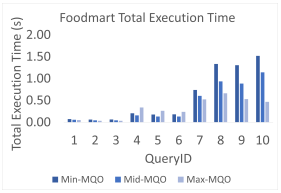
(k) TPC-DS 10M *Time* Workload Facilitator Queries Breakdown



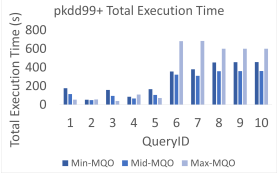
(l) TPC-DS 100M *Time* Workload Facilitator Queries Breakdown



(m) Northwind Workload



(n) Foodmart Time Workload



(o) pkdd99+ Time Workload

Figure 6: Performance Evaluation Figures

performance evaluation for each dataset with respect to the total execution time of each algorithm. For the TPC-DS *Item* and *Time* workloads, Figure 6 also presents the facilitator queries' breakdown for each ANALYZE query of the workload.

6.4.1 TPC-DS *Item* Workload

Figures 6a, 6b, and 6c illustrate the *Item* query workload's execution time across all methods for each TPC-DS dataset size variation. Min-MQO is shown as a dark blue line, Mid-MQO as a blue line, Max-MQO as a light blue line. The dimension tables involved in the workload are the *Date* dimension (≈80000 tuples) and the *Item* dimension (≈ 20000 tuples). We observe the following:

- Regardless of the dataset size, Mid-MQO performs the best.
- Max-MQO constantly performs the worst, especially when the selectivity ratio is increasing.
- Min-MQO and Mid-MQO do not differ significantly in execution time.

Figures 6d,6e,6f provide the detailed execution time for each facilitator query of the workload's ANALYZE queries. We observe that the drill-down queries that Mid-MQO does not execute in comparison to Min-MQO, are not very significant for the total execution time. Thus, the difference in performance in Min-MQO and Mid-MQO is relatively small. Max-MQO processes a large number of tuples compared to the other antagonists and, in that context, underperforms constantly.

6.4.2 TPC-DS *Time* Workload

Figures 6g,6h,6i illustrate the *Time* query workload's execution time across all methods for each TPC-DS dataset size variation. The dimension tables involved in the workload are the *Date* dimension (≈80000 tuples) and the *Time* dimension (≈ 70000 tuples). We observe the following.

- In the TPC-DS 2M, Max-MQO outperforms the other algorithms, and Min-MQO is the worst performing algorithm in every case.
- In the TPC-DS 10M and 100M, in small selectivity ratios, Max-MQO performance is almost stable regarding the selectivity ratio increase, while Min-MQO's and Mid-MQO's performance deteriorates as the selectivity ratio increases.

Here, due to the large size of both dimension tables, the performance of Max-MQO is stable, as it consistently explores a large subset of the fact table once, and avoids the heavy extra cost of siblings of the other algorithms when the selectivity increases (see Figures 6j, 6k, 6l for the breakdown, especially for high-selectivity queries with large QueryID's). The breakdown of the execution time in facilitator queries is presented in Figure 6j, 6k, 6l showing that as the

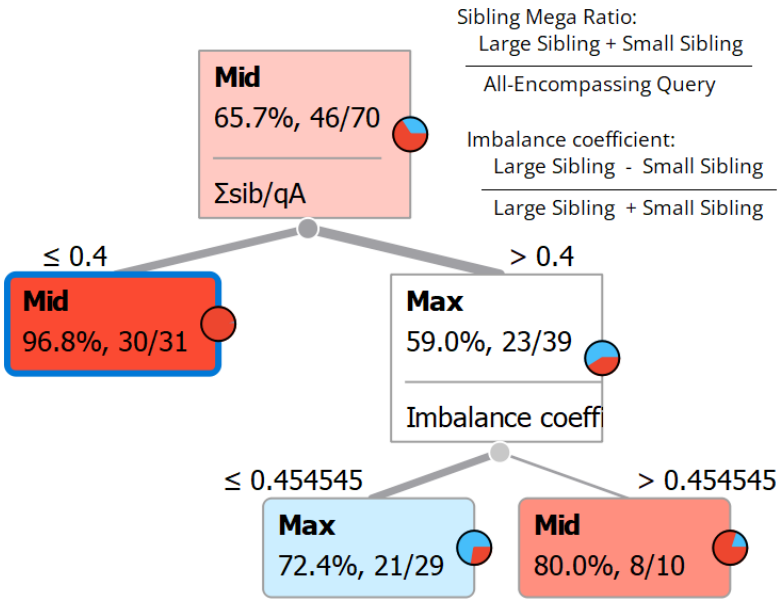


Figure 7: Optimal Algorithm Criteria

selectivity ratio increases, the execution cost of the drill-down queries justifies the increase in the total execution time difference between Min-MQO and Mid-MQO, thus increasing the performance difference between Min-MQO and Mid-MQO as well.

6.4.3 Other datasets

Finally, when comparing the algorithms on the 3 other datasets (Figure 6m, 6n, 6o) we observe.

- In the Northwind workload (Table 9) that involves two small dimension tables (≈90 tuples and ≈9 tuples), Mid-MQO constantly scores wins against the other antagonists, while Max-MQO is underperforming.
- In the Foodmart workload (Table 8) that involves unbalanced dimension tables (≈10000 tuples and ≈1000 tuples), Max-MQO is the fastest algorithm, while Min-MQO is the slowest.
- In the pkdd99+ workload (Table 7) that involves one fairly large and one small dimension (≈30000 tuples and ≈5000), Mid-MQO outperforms the other antagonists in the high selectivity ratios, while Max-MQO performs better in the low selectivity ratios.

6.5 Effect of Facilitator Queries Performance

Figures 6d,6e,6f present the execution time for all five facilitator queries for the TPC-DS *Item* workload. We observe that Min-MQO's performance closely follows Mid-MQO's performance regardless of dataset size (Figures 6a,6b,6c). The drill-down queries' execution time for the TPC-DS Item workload is a small percentage of the total execution time of the facilitator queries. However, the execution time of the siblings is a very large percentage of the total execution time. Since both algorithms execute the sibling queries and Mid-MQO omits the drill-down's execution, it is evident that the performance of Min-MQO will follow the performance of Mid-MQO.

Figures 6j, 6k, 6l illustrate the execution time for all five facilitator queries for the TPC-DS *Time* workload. In the case of 2 million facts, the performance gap between Min-MQO and Mid-MQO is significant. We observe that the drill-down queries' execution time is almost 1/3 of the total execution time. In this context, we can state that Mid-MQO which does not execute the drill-down queries, gains a performance advantage over Min-MQO. In all other data sizes (10 million and 100 million), the execution time of the drill-down's is not as large as in the 2 million case and the performance gap between Min-MQO and Mid-MQO is shrinking.

Taking into account the aforementioned observations, we can state that the execution time of the drill-down queries can affect the performance gap between Min-MQO and Mid-MQO. If the execution time of the drill-down queries is a significant percentage of the total execution time, then Mid-MQO greatly outperforms Min-MQO. However, if the execution time of the drill-down queries is not as significant, then Min-MQO is outperformed by Mid-MQO but follows Mid-MQO's performance closely.

When does this happen? The crux of the distinction does not lie at the drill down queries, but rather at the extent of the sibling queries. The space of detailed fact tuples that the drill down queries explore is exactly the same with the one of the original query; it is only the eventual aggregation levels that differ. On the other hand, depending on the siblings involved, sibling queries can span a small subset of the detailed facts, or, a very large one. Thus, their ratio over the drill down queries changes too.

6.6 Choosing Algorithms

Mid-MQO is a safe choice as a query processing strategy, as it is both stable and winning most of the time. However, can we do better than this first result? We have analyzed the entire corpus of 70 workloads (Tables 5, 6, 7) of the large TPC-DS and pkdd99+ datasets (7 workloads of 10 queries each) to *precisely detect* when each query processing strategy wins, via easily computable cost measures. Ultimately (Figure 7), *Max-MQO wins when (a) the siblings touch a fairly large number of fact tuples compared to the all-encompassing query (more than 40%), and, (b) there is a fairly small imbalance between the sibling queries (less than 45%). In all other cases, Mid-MQO wins.* The rationale is simple:

1. when q^A is too large compared to facilitator queries, then, it explores too much of the data space, hence Max-MQO loses.

2. when the siblings are both sizable and imbalanced, they do not significantly overlap, thus Max-MQO, visiting them once, does not save time.

The only case where Max-MQO saves time is if sizable siblings overlap (estimated via the imbalance coefficient). Naturally, the exact thresholds are an engineering default, subject to a wider evaluation over time; however, the rationale is consistent with the theoretical setup of q^A.

7 Conclusions

The paper introduces ANALYZE, a novel intentional cube query operator that provides a 360° view of the data by combining original, sibling, and drill-down queries in a single invocation with formal semantics. We have shown that the operator internal queries can be safely merged with theoretical correctness guaranties, enabling principled multi-query optimization at the operator level without modifying the underlying query optimizer. We have introduced, implemented in a data analytics system, and extensively evaluated three execution strategies, demonstrating that partial merging (Mid-MQO) delivers robust and scalable performance across workloads, and full merging (Max-MQO) is beneficial only when the siblings are sizable and significantly overlap, while the not-optimized strategy (Min-MQO) is consistently suboptimal.

As part of future work, several paths can be followed. Right now, a plain DBMS would simply execute Min-MQO to process the set of facilitator queries of the operator. A lesson learned here is that the external data analytics engine *can* optimize query execution *outside the DBMS*. This is a lesson to be applied to future intentional operators as well (PREDICT, EXPLAIN, etc). Exploiting the formal, algebraic nature of the operator also raises the question of integrating it with cost-based DBMS optimizers or platforms like Apache Spark. Several possible extensions concern the internal design choices of the current version of ANALYZE. Relaxing some of the constraints (e.g., by more groupers, siblings based on user-defined or context-dependent benchmarks [SPV07, MB25]) is also open to research. For example, generalizing our results to more than two groupers is possible; we conjecture that despite the formalization overhead that this incurs, there is no limit to the number of groupers that can be involved in the formulation of an ANALYZE query. As another example, regulating or even automating the benchmark against which the original query is contrasted is also possible (e.g., comparing sales of a quarter to the same quarter of a previous year), with the respective impact to the query processing algorithms.

Acknowledgments. The research project is implemented in the framework of H.F.R.I call "3rd Call for H.F.R.I.'s Research Projects to Support Faculty Members & Researchers" (H.F.R.I. Project Number: 23640).

D. Gkitsakis helped with early, trial versions of the code.

8 Artifacts

The source code, data and/or other artifacts have been made available at `https://github.com/DAINTINESS-Group/DelianCubeEngine`. Navigate to the package analyzemqoexperiments at `https://tinyurl.com/bddf4tzv` where there is a markdown file with directions on how to repeat the experiments.

References

[AKS+21] Firas Abuzaid, Peter Kraft, Sahaana Suri, Edward Gan, Eric Xu, Atul Shenoy, Asvin Ananthanarayan, John Sheu, Erik Meijer, Xi Wu, Jeffrey F. Naughton, Peter Bailis, and Matei Zaharia. DIFF: a relational interface for large-scale data explanation. *VLDB J.*, 30(1):45–70, 2021.

[Ame24] Sihem Amer-Yahia. Intelligent agents for data exploration. *Proc. VLDB Endow.*, 17(12):4521–4530, 2024.

[AMP23] Sihem Amer-Yahia, Patrick Marcel, and Verónika Peralta. Data narration for the people: Challenges and opportunities. In *Proceedings 26th International Conference on Extending Database Technology, EDBT 2023, Ioannina, Greece, March 28-31, 2023*, pages 855–858. OpenProceedings.org, 2023.

[BRH+22] Tijl De Bie, Luc De Raedt, José Hernández-Orallo, Holger H. Hoos, Padhraic Smyth, and Christopher K. I. Williams. Automating data science. *Commun. ACM*, 65(3):76–87, 2022.

[Del25] Delian Cubes. `https://github.com/DAINTINESS-Group/DelianCubeEngine`, 2025. DAINTINESS GROUP University of Ioannina.

[DHX+19] Rui Ding, Shi Han, Yong Xu, Haidong Zhang, and Dongmei Zhang. QuickInsights: Quick and automatic discovery of insights from multi-dimensional data. In *Proceedings of SIGMOD*, pages 317–332, 2019.

[EMS19] Ori Bar El, Tova Milo, and Amit Somech. ATENA: an autonomous system for data exploration based on deep reinforcement learning. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM 2019, Beijing, China, November 3-7, 2019*, pages 2873–2876, 2019.

[FGM+21] Matteo Francia, Matteo Golfarelli, Patrick Marcel, Stefano Rizzi, and Panos Vassiliadis. Assess queries for interactive analysis of data cubes. In *Proceedings of the 24th International Conference on Extending Database Technology, EDBT 2021, Nicosia, Cyprus, March 23 - 26, 2021*, pages 121–132, 2021.

[FGM+23] Matteo Francia, Matteo Golfarelli, Patrick Marcel, Stefano Rizzi, and Panos Vassiliadis. Suggesting assess queries for interactive analysis of multidimensional data. *IEEE Trans. Knowl. Data Eng.*, 35(6):6421–6434, 2023.

[FMPR22] Matteo Francia, Patrick Marcel, Verónika Peralta, and Stefano Rizzi. Enhancing cubes with models to describe multidimensional data. *Inf. Syst. Frontiers*, 24(1):31–48, 2022.

[FRM24] Matteo Francia, Stefano Rizzi, and Patrick Marcel. Explaining cube measures through intentional analytics. *Inf. Syst.*, 121:102338, 2024.

[GV13] Dimitrios Gkesoulis and Panos Vassiliadis. Cinecubes: cubes as movie stars with little effort. In *Proceedings of the sixteenth international workshop on Data warehousing and OLAP, DOLAP 2013, San Francisco, CA, USA, October 28, 2013*, pages 3–10, 2013.

[GVM15] Dimitrios Gkesoulis, Panos Vassiliadis, and Petros Manousis. Cinecubes: Aiding data workers gain insights from OLAP queries. *Inf. Syst.*, 53:60–86, 2015.

[HRK+09] Mingsheng Hong, Mirek Riedewald, Christoph Koch, Johannes Gehrke, and Alan J. Demers. Rule-based multi-query optimization. In *EDBT 2009, 12th International Conference on Extending Database Technology, Saint Petersburg, Russia, March 24-26, 2009, Proceedings*, pages 120–131, 2009.

[Hyd25] Julian Hyde. Foodmart. `https://github.com/julianhyde/foodmart-data-hsqldb`, 2025. Accessed: 2025-07-24.

[IPC15] Stratos Idreos, Olga Papaemmanouil, and Surajit Chaudhuri. Overview of data exploration techniques. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 277–281, 2015.

[KS15] Tarun Kathuria and S. Sudarshan. Greedy awakens : Efficient and provable multi-query optimization. *CoRR*, abs/1512.02568, 2015.

[LKDL12] Wangchao Le, Anastasios Kementsietsidis, Songyun Duan, and Feifei Li. Scalable multi-query optimization for SPARQL. In *IEEE 28th International Conference on Data Engineering (ICDE 2012), Washington, DC, USA (Arlington, Virginia), 1-5 April, 2012*, pages 666–677, 2012.

[LMS+25] Tavor Lipman, Tova Milo, Amit Somech, Tomer Wolfson, and Oz Zafar. LINX: A language driven generative system for goal-oriented automated data exploration. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025,*

Barcelona, Spain, March 25-28, 2025, pages 270–283. OpenProceedings.org, 2025.

[LYZ+23] Haotian Li, Lu Ying, Haidong Zhang, Yingcai Wu, Huamin Qu, and Yun Wang. Notable: On-the-fly assistant for data storytelling in computational notebooks. In *CHI*, 2023.

[MB25] Pablo Mateos and Alejandro Bellogín. A systematic literature review of recent advances on context-aware recommender systems. *Artif. Intell. Rev.*, 58(1):20, 2025.

[MDHZ21] Pingchuan Ma, Rui Ding, Shi Han, and Dongmei Zhang. Metainsight: Automatic discovery of structured knowledge for exploratory data analysis. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, pages 1262–1274. ACM, 2021.

[MDW+23] Pingchuan Ma, Rui Ding, Shuai Wang, Shi Han, and Dongmei Zhang. Insightpilot: An llm-empowered automated data exploration system. In *EMNLP'2023*, 2023.

[Mic23] Microsoft. Northwind. `https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs`, 2023. Accessed: 2025-09-14.

[MS20] Tova Milo and Amit Somech. Automating exploratory data analysis via machine learning: An overview. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, pages 2617–2622, 2020.

[PAB+21] Aurélien Personnaz, Sihem Amer-Yahia, Laure Berti-Équille, Maximilian Fabricius, and Srividya Subramanian. DORA THE EXPLORER: exploring very large data with interactive deep reinforcement learning. In *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*, pages 4769–4773, 2021.

[RS09] Prasan Roy and S. Sudarshan. Multi-query optimization. In *Encyclopedia of Database Systems*, pages 1849–1852. Springer US, 2009.

[SAM98] Sunita Sarawagi, Rakesh Agrawal, and Nimrod Megiddo. Discovery-driven exploration of OLAP data cubes. In *Advances in Database Technology - EDBT'98, 6th International Conference on Extending Database Technology, Valencia, Spain, March 23-27, 1998, Proceedings*, pages 168–182, 1998.

[Sar99] Sunita Sarawagi. Explaining differences in multidimensional aggregates. In *VLDB'99, Proceedings of 25th International Conference on Very Large Data Bases, September 7-10, 1999, Edinburgh, Scotland, UK*, pages 42–53, 1999.

[Sar00] Sunita Sarawagi. User-adaptive exploration of multidimensional data. In *VLDB 2000, Proceedings of 26th International Conference on Very Large Data Bases, September 10-14, 2000, Cairo, Egypt*, pages 307–316, 2000.

[SCC+23] Mengdi Sun, Ligan Cai, Weiwei Cui, Yanqiu Wu, Yang Shi, and Nan Cao. Erato: Cooperative data story editing via fact interpolation. *IEEE Trans. Vis. Comput. Graph.*, 29(1):983–993, 2023.

[Sel88] Timos K. Sellis. Multiple-query optimization. *ACM Trans. Database Syst.*, 13(1):23–52, 1988.

[SG90] Timos K. Sellis and Subrata Ghosh. On the multiple-query optimization problem. *IEEE Trans. Knowl. Data Eng.*, 2(2):262–266, 1990.

[SPV07] Kostas Stefanidis, Evaggelia Pitoura, and Panos Vassiliadis. A context-aware preference database system. *Int. J. Pervasive Comput. Commun.*, 3(4):439–460, 2007.

[SS01] Gayatri Sathe and Sunita Sarawagi. Intelligent rollups in multidimensional OLAP data. In *VLDB 2001, Proceedings of 27th International Conference on Very Large Data Bases, September 11-14, 2001, Roma, Italy*, pages 531–540, 2001.

[SXS+21] Danqing Shi, Xinyue Xu, Fuling Sun, Yang Shi, and Nan Cao. Calliope: Automatic visual data story generation from a spreadsheet. *IEEE Trans. Vis. Comput. Graph.*, 27(2):453–463, 2021.

[THY+17] Bo Tang, Shi Han, Man Lung Yiu, Rui Ding, and Dongmei Zhang. Extracting top-k insights from multi-dimensional data. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*, pages 1509–1524, 2017.

[TPC24] TPC. Tpc-ds. `https://www.tpc.org/tpcds/`, 2024. Accessed: 2025-06-05.

[Vas22] Panos Vassiliadis. A Cube Algebra with Comparative Operations: Containment, Overlap, Distance and Usability. *CoRR*, abs/2203.09390, 2022.

[Vas23] Panos Vassiliadis. Cube query answering via the results of previous cube queries. In *Proceedings of the 25th International Workshop on Design, Optimization, Languages and Analytical Processing of*

Big Data (DOLAP) co-located with the 26th International Conference on Extending Database Technology and the 26th International Conference on Database Theory (EDBT/ICDT 2023), Ioannina, Greece, March 28, 2023, pages 71–75. CEUR-WS.org, 2023.

[VM18] Panos Vassiliadis and Patrick Marcel. The road to highlights is paved with good intentions: Envisioning a paradigm shift in OLAP modeling. In *Proceedings of the 20th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data co-located with 10th EDBT/ICDT Joint Conference (EDBT/ICDT 2018), Vienna, Austria, March 26-29, 2018*, 2018.

[VMR19] Panos Vassiliadis, Patrick Marcel, and Stefano Rizzi. Beyond roll-up's and drill-down's: An intentional analytics model to reinvent OLAP. *Information Systems*, 85:68–91, 2019.

[WC13] Guoping Wang and Chee-Yong Chan. Multi-query optimization in mapreduce framework. *Proc. VLDB Endow.*, 7(3):145–156, 2013.

[WSZ+20] Yun Wang, Zhida Sun, Haidong Zhang, Weiwei Cui, Ke Xu, Xiaojuan Ma, and Dongmei Zhang. Datashot: Automatic generation of fact sheets from tabular data. *IEEE Trans. Vis. Comput. Graph.*, 26(1):895–905, 2020.

[XWJ24] Junjie Xing, Xinyu Wang, and H. V. Jagadish. Data-driven insight synthesis for multi-dimensional data. *VLDB Endow.*, 17(5):1007–1019, 2024.

[ZYO99] Ning Zhong, Yiyu Yao, and Setsuo Ohsuga. Peculiarity oriented multi-database mining. In *Principles of Data Mining and Knowledge Discovery, Third European Conference, PKDD '99, Prague, Czech Republic, September 15-18, 1999, Proceedings*, volume 1704 of *Lecture Notes in Computer Science*, pages 136–146. Springer, 1999.